

Quasaria: Mainnet Launch Audit & Readiness Checklist

Stellar DEX: 7 Soroban contracts, keeper bot, React/Vite frontend

Prepared Saturday, Sep 26, 2026 (Arizona time, MST/UTC-7) at commit 2c2e803

- [How to read this document](#)
- [0. Evidence gathered and tool results](#)
 - [0.1 Scope reviewed](#)
 - [0.2 Tools run \(all on the box, Sep 26, 2026\)](#)
 - [0.3 Wasm hashes: local build = testnet on-chain code](#)
 - [0.4 Live testnet state \(read-only, ledger 4,885,016, Sep 26, 2026 11:51 AM MST\)](#)
 - [0.5 Proof-of-concept results \(scratch copy /tmp/poc , not committed\)](#)
 - [0.6 Findings from this review](#)
- [1. Smart-contract security](#)
- [2. Economic and tokenomics review](#)
- [3. Oracle](#)
- [4. Liquidity, staking and pool contracts](#)
- [5. Access control and admin keys](#)
- [6. Frontend and wallet integration](#)
- [7. Bot / keeper](#)
- [8. Deployment and readiness](#)
- [9. Legal, compliance and business](#)
- [10. Funding plan \(90-day launch runway\)](#)
- [11. Additional launch items](#)
- [12. Executive summary](#)
 - [12.1 Verdict: **NO-GO for mainnet**](#)
 - [12.2 Status counts](#)
 - [Full item index](#)
 - [12.3 Blockers in priority order](#)
 - [12.4 Suggested order of work](#)
 - [12.5 External audit: typical industry ranges \(NOT quotes\)](#)
 - [12.6 Limits of this review](#)

What this is: an internal readiness review and launch checklist. It is **not** an external security audit and it is **not** legal advice. Every status below is based on evidence gathered on Sep 26, 2026 from the repository at `/workspace/stellar-dex` (commit `2c2e803`, public repo `github.com/yogibear1323/quasaria`), read-only queries of the live **testnet** deployment, and tools run locally. Nothing was deployed, no funds were spent, mainnet was not touched, and no code was changed in the repository. Proof-of-concept tests were run in a scratch copy under `/tmp/poc` and are not in the repo.

Verdict at a glance: NO-GO for mainnet. 57 of the 86 checklist items block mainnet today (status counts: Pass 4, Partial 35, Fail 13, Not started 33, Unknown 1). There has been no external audit, the review found confirmed contract bugs (with PoCs), one hot key controls every contract and there is no multisig, timelock or pause, the oracle is a mock, and the legal and compliance work has not started. The full executive summary is at the end ([§12](#)).

How to read this document

Each checklist item has six fields:

- **Covers:** what the item is about.
- **Pass looks like:** the objective bar for “done”.
- **Evidence that should exist:** the report or artifact a reviewer should be able to open.
- **Current status:** one of **Pass**, **Partial**, **Fail**, **Not started** or **Unknown**, with the evidence (file paths, line numbers, commands, live reads).
- **Blocks mainnet:** **YES** or **NO**.

Findings from this review are numbered **F-01...F-21** in [§0.6](#) and referenced from the checklist. Severity uses the usual Critical / High / Medium / Low scale. Severity is this reviewer’s judgement and must be re-rated by the external auditor.

All times are Arizona time (MST, UTC-7). On-chain timestamps were converted from UTC.

0. Evidence gathered and tool results

0.1 Scope reviewed

Area	Files (non-test lines / nSLOC*)
QFX token (1:1 XLM wrapper + holder yield)	<code>contracts/reward-token/src/lib.rs</code> (840 / 664)
AMM pool ($x*y=k$, SEP-41 LP token)	<code>contracts/amm-pool/src/lib.rs</code> (624 / 542)
Router	<code>contracts/router/src/lib.rs</code> (132 / 107)
Staking (“Orbit”)	<code>contracts/staking/src/lib.rs</code> (350 / 294)
Referral registry	<code>contracts/referral/src/lib.rs</code> (217 / 168)

Area	Files (non-test lines / nSLOC*)
Leverage vault (“Warp”)	contracts/leverage-vault/src/lib.rs (686 / 567)
Mock oracle	contracts/mock-oracle/src/lib.rs (112 / 86)
Build/deploy/seed scripts	scripts/build.sh, scripts/deploy-testnet.sh, scripts/migrate-qfx-peg-testnet.sh, scripts/seed-testnet.sh, scripts/seed-stablecoin-pools.{sh,ts}, .github/workflows/pages.yml
Bot / keeper	bot/src/*.ts (config, keeper, soroban, index, risk, math, strategies)
Frontend key handling & tx paths	frontend/src/lib/{keys,signer,wallet,config,soroban,stellar}.ts(x), frontend/src/pages/{Trade,Pools,Bots,Stake,Referrals,Landing}.tsx, frontend/index.html, frontend/vite.config.ts
Deploy config	deployments/testnet.json, deployments/testnet-stablecoins.json, frontend/src/config/testnet.json, bot/.env.example

*nSLOC = non-blank, non-comment lines, excluding `#[cfg(test)]/mod test;`, counted with `grep` on the box. Total production contract code is **2,428 nSLOC**, or about 2,340 without the mock oracle.

0.2 Tools run (all on the box, Sep 26, 2026)

Tool	Command	Result
Unit tests	<code>cargo test --workspace</code> (rustc/cargo 1.98.1, soroban-sdk 28.0.0)	41 passed, 0 failed. By crate: reward-token 14, leverage-vault 7, amm-pool 6, staking 6, referral 5, router 2, mock-oracle 1.
Coverage	<code>cargo llvm-cov --workspace --summary-only</code> (cargo-llvm-cov 0.9.1)	Production <code>lib.rs</code> line coverage 88.8% (1,582/1,781 lines). amm-pool 72.3% , mock-oracle 79.6%, router 91.9%, leverage-vault 92.6%, staking 95.4%, reward-token 95.7%, referral 96.4%. Whole workspace including tests: 92.5% lines / 93.2% regions.
Clippy (default)	<code>cargo clippy --workspace --all-targets</code>	11 warnings, 0 errors, no correctness lints. 7 × <code>needless_borrows_for_generic_args</code> (e.g. <code>amm-pool/src/lib.rs:527</code> , <code>reward-token/src/lib.rs:537,565</code> , <code>staking/src/lib.rs:227,248</code> , <code>leverage-vault/src/lib.rs:392,415</code>), 2 × <code>manual_range_contains</code> (tests), 1 × <code>needless_late_init</code> (<code>leverage-vault/src/lib.rs:310</code>).
Clippy (strict, arithmetic)	<code>cargo clippy --lib -- -W clippy::arithmetic_side_effects -W clippy::integer_division -W clippy::unwrap_used -W clippy::cast_possible_truncation -W clippy::cast_sign_loss -W clippy::cast_possible_wrap</code>	113 unchecked arithmetic ops, 30 integer divisions, 24 <code>unwrap()</code> , 3 truncating casts (<code>leverage-vault/src/lib.rs:289,462</code> , <code>reward-token/src/lib.rs:727</code>), 1 sign-loss cast. By file: vault 63, reward-token 45, amm-pool 39, staking 25, referral 4, router 2, oracle 1. Mitigation: <code>overflow_checks = true</code> in <code>[profile.release]</code> (<code>contracts/Cargo.toml</code>), so overflow panics and never wraps. Panics are still DoS vectors (see F-04).

Tool	Command	Result
Rust dependency audit	<code>cargo audit</code> (cargo-audit 0.22.2, 1,271 advisories, 221 crates)	0 vulnerabilities. 1 <i>unmaintained</i> warning: paste 1.0.15 (RUSTSEC-2024-0436), a transitive dependency of soroban-sdk.
Scout (CoinFabrik)	<code>cargo scout-audit v0.3.17</code> with detectors nightly-2025-09-18	Ran successfully (after 4 failed attempts: the remote detector download failed with “Could not find suitable branch”; the older scout-soroban detectors are pinned to nightly-2024-07-11, which can’t build soroban-env-host (needs rustc ≥ 1.91); OpenSSL env vars were needed; and a nested <code>cargo install</code> deadlocked on a shared <code>CARGO_TARGET_DIR</code>). All 7 crates analyzed. Scout’s per-crate totals: 111 Critical, 51 Medium, 0 Minor, 42 Enhancement . Critical = unchecked arithmetic only (35 subtraction, 34 addition, 32 generic, 7 negation, 3 multiplication), the same set as <code>clippy arithmetic_side_effects</code> , which <code>overflow-checks = true</code> turns into panics (see F-04, F-02). Medium: 24 <code>unwrap</code> ; dynamic storage / unguarded Vec push at <code>leverage-vault/src/lib.rs:242,253,529</code> (confirms F-02); missing new-admin auth at <code>reward-token/src/lib.rs:632</code> (single-step <code>set_admin</code> , \$5.7); LP-token approve/burn/burn_from emit no SEP-41 events at <code>amm-pool/src/lib.rs:566,603,607</code> (new F-21); division-before-multiplication at <code>amm-pool:143</code> , <code>reward-token:241</code> (the overflow-fallback branch of <code>mul_div</code> , <code>(a/c)*b + (a%c)*b/c</code> , which is exact: false positive); front-running on <code>amm-pool:320,501,502</code> (false positive in the contracts, which enforce <code>min_out/min_a/min_b</code> , but true for the frontend, which sends 0: F-10); unbounded loop at <code>router:108</code> (false positive, bounded by <code>MAX_HOPS = 4</code>); extend_ttl equal-argument notes at <code>amm-pool:584</code> , <code>reward-token:776</code> (allowance TTL = expiry, intended); QFX token-event warnings (false positive: events are emitted in helpers). Enhancement: 42 × “emit an event when storage is modified” (e.g. <code>vault fund_liquidity :386</code> , <code>liquidate :576</code> , all admin setters). Raw report: <code>/tmp/scout-report.md</code> (not committed).
npm audit (frontend)	<code>npm audit in frontend/</code>	0 vulnerabilities (0 critical / 0 high / 0 moderate / 0 low). 125 dependencies (57 prod, 69 dev, 27 optional). <code>--omit=dev: 0</code> .
npm audit (bot)	<code>npm audit in bot/</code>	0 vulnerabilities. 132 dependencies (42 prod, 91 dev). <code>--omit=dev: 0</code> .
JS tests	<code>npx vitest run</code>	frontend 79/79 passed (6 files). bot 20/20 passed (4 files). bot: <code>tsc --noEmit</code> clean.
Secret scan	<code>git log -p --all + working tree + frontend/dist</code> scanned for Ed25519 secret-seed strings, validated with <code>StrKey.isValidEd25519SecretSeed</code>	0 valid secret seeds found. <code>bot/.env</code> exists but is git-ignored (<code>.gitignore:15 *.env</code>) and holds no secret (only a commented placeholder).

Tool	Command	Result
Reproducible build	Rebuilt with stellar-cli 28.0.0 (release tarball sha256 2075444867...0ce3fdc, matches the GitHub release digest) + rustc 1.98.1: stellar contract build - -out-dir /tmp/qwasm	All 7 wasm hashes match the committed-workspace artifacts. See §0.3.
On-chain code verification	stellar contract fetch for every contract in deployments/testnet.json (read-only)	All 8 active testnet contracts run exactly the wasm built from HEAD (both pools share one hash). Legacy QFX CDUMMY...E774HZ runs a different, older wasm that still exports mint (§0.3).
Live testnet state	Read-only simulateTransaction on RPC soroban-testnet.stellar.org at ledger 4,885,016 (Sep 26, 2026 11:51 AM MST)	See §0.4.
PoC tests (scratch copy)	/tmp/poc, cargo test audit_poc - --nocapture	5 PoCs, all reproduced: F-01, F-02, F-03, F-04, F-07 (§0.5).

0.3 Wasm hashes: local build = testnet on-chain code

Contract	Testnet ID	sha256 (local build == on-chain)
QFX (quasaria_reward_token.wasm)	CBHU04V3...WJFBX66	39e57ecd32ebb070c55635b94d775fc88dde783d41b9f97442fec13720a33e54
AMM pool (XLM/QUSD and QFX/QUSD)	CB4H0L3D...EP7Q0V7F, CCT64AIV...BZPBEVIV	95214a0b939b377dc94a0217ccfd1d9c45e3c72292fbfd3e972471c3d51fe6ce
Router	CDVMF4C3...AAIRGXRCIK	e6fb074bf5df97bdf40e25a4612c277e398551a4328492d1e20377f48dc7353d
Staking	CCYPZCFR...KGQAQF400	c73417d4d10410c339c2bfe7047a61ff57d35fe618d3752af1446a2baelabb45
Referral	CBUK04MS...3AAH760UJ3	e6a2a57a17c0e2695759e75fb9de6efcb91f4a585b9583021fb6a9659c651cf7
Leverage vault	CDBGDS5K...LTICNKZNF	a73f130216442ba11858bdcd7625a97a7e75be9bcea694e100159af33e60a516
Mock oracle	CBE3R07H...26HSEVJMPK	819620147fed705c09c8000e925dd1374f1dffc0acd197000bfa6faebbae5c2f
Legacy QFX (retired, testnet)	CDUMMYBM...CHN2E774HZ	2a266c15...a6ffb7e398: different code; exports admin mint

Contract meta embedded in the on-chain QFX wasm reads: network: testnet-only scaffold, unaudited, rsver 1.98.1, rssdkver 28.0.0#48d5067, cliver 28.0.0#300aaf6.

0.4 Live testnet state (read-only, ledger 4,885,016, Sep 26, 2026 11:51 AM MST)

Read	Value	Meaning
qfx.reserves() ()	xlm_reserve 8,500 = total_supply 8,500, surplus 0, fully_backed true , reward_reserve 2,500, circulating 6,000	Peg holds on testnet.
qfx.yield_info() fo()	apr 1200 bps (APY 1274), eligible_supply 300 QFX , reward_pool 2,500, daily_emission 0.0986 QFX	Runway looks huge only because eligible supply is tiny.
staking.pool(0) QFX→QFX	rate 1000 stroops/s (8.64 QFX/day), reserve 1,498.39 QFX, staked 200, lock 7 d	Runway ≈ 173 days at the current rate.
staking.pool(1) QLP→QFX	rate 2000 stroops/s (17.28 QFX/day), reserve 1,496.79, staked 100	Runway ≈ 86.6 days (< 90).
vault.config() ()	max_leverage 10x (100000 bps), maintenance 5%, liq. bonus 5%, open fee 0.10%, max_price_age 900 s	The landing page advertises 20x (F-15).
vault.liquidity() ity()	200,000.52 QUSD	Admin-issued demo stablecoin.
vault.open_position_ids())	[1, 2]	
oracle.lastprice(XLM) rice(XLM)	0.1234 USD, timestamp 1790385877 = Sep 25, 2026 6:24 PM MST, 17.4 h old	Older than max_price_age (900 s). Every vault open/close/liquidate reverts with StalePrice right now , so liquidations are impossible (F-06).
Pools	XLM/QUSD 5,199.88 XLM / 576.99 QUSD; QFX/QUSD 2,500 QFX / 277.41 QUSD; fee 30 bps	
referral.share_bps() re_bps()	2000 (20% of fees)	
Admin account (Horizon)	GDAEZGA6...FB3VBNKCY: one signer (master, weight 1), thresholds 0/0/0	Single hot key controls every contract (F-09).
Legacy QFX total_supply() ()	5,000,000 QFX, unbacked	Legacy contract still has admin mint (F-12).

0.5 Proof-of-concept results (scratch copy /tmp/poc, not committed)

PoC	Output (verbatim)
Vault dust-position spam (F-02)	5000 dust positions cost the attacker 5000 stroops total; OpenIds XDR = 60012 bytes, then honest open_position after spam: cpu=24852808 mem=14054149
QFX boundary yield sniping (F-03)	whale held 120 s across the boundary and extracted 3287671232 stroops (328.7671232 XLM) of reserve yield (1,000,000 XLM deposited 60 s before the UTC-day boundary, redeemed 60 s after)

PoC	Output (verbatim)
Staking accumulator overflow (F-04)	acc_reward_per_share after 1-unit stake = 100000000000000000000000000000000, then victim stake of 10,000 tokens -> true (reverted: overflow panic)
Admin drains vault reserve (F-01)	after admin withdrew 1000000000000 reserve, a +500% PnL trade paid out 1005000000 (margin only) (the extra 5,000,000 is the trader's own open fee)
Future oracle timestamp (F-07)	position 1 opened on a price stamped 30 days in the future, 20 days after it was pushed (max_price_age = 600 s)

0.6 Findings from this review

These are internal-review findings, not auditor-confirmed. “PoC” means reproduced in the scratch test copy (\$0.5).

ID	Sev.	Contract / area	Finding (evidence)
F-01	Critical	Leverage vault: admin powers	The admin can pull the entire counterparty reserve at any time with <code>withdraw_liquidity(to, amount)</code> (leverage-vault/src/lib.rs:396-405) and can swap the oracle instantly with <code>set_oracle</code> (:367-370). There is no timelock and no multisig. Trader profit is silently capped by whatever liquidity is left (<code>settle</code> , :307-321), so winners can go unpaid. Third parties can deposit through <code>fund_liquidity</code> (“anyone can add; only admin removes”, :385-394), which makes those deposits admin-withdrawable. PoC.
F-02	High	Leverage vault: unbounded storage / DoS	Every open rewrites one global <code>OpenIds</code> vector (:528-530, <code>put_list</code> :241-246). There is no minimum margin. With <code>margin = 1</code> stroop the fee rounds to 0 (:475-476) and maintenance rounds to 0, so <code>HF = i128::MAX</code> (:179-182) and the position can never be liquidated . At 12 bytes per id, <code>OpenIds</code> reaches the network's 65,536-byte contract-data entry limit at about 5,460 open positions . From then on every <code>open_position</code> for every user reverts, and the attacker's dust can only be closed by the attacker. Cost to the attacker: about 5,460 stroops of margin plus tx fees. Before that point every user's CPU cost, write size and fees grow linearly. PoC: 5,000 positions give a 60,012-byte list and 24.9M instructions per honest open. The keeper also scans $O(N)$.
F-03	High	QFX: yield timing	Holder yield accrues per whole UTC day, on the eligible balance present at <code>accrual</code> (<code>projected</code> , <code>reward-token/src/lib.rs</code> :339-361). A deposit made just before the day boundary earns a full day , and can be redeemed right after. PoC: 1,000,000 XLM held for 120 s extracted 328.77 XLM from the reserve. The reserve drains according to peak balances at the boundary, not time-weighted ones.
F-04	High	Staking: precision/overflow DoS	<code>acc_reward_per_share += reward * 1e18 / total_staked</code> (<code>staking/src/lib.rs</code> :153) and there is no minimum stake. A 1-unit staker in an empty pool pushes <code>acc</code> to <code>1e28</code> . After that, any stake above $\sim 1.7e10$ units overflows <code>pos.amount * acc</code> (:161, :256), panics (because <code>overflow-checks = true</code>), and bricks the pool for new stakers . PoC.
F-05	High	AMM: admin drain path	<code>set_referral</code> (<code>amm-pool/src/lib.rs</code> :372-378) lets the admin point a pool at any contract. The pool trusts that contract's <code>share_bps()</code> without re-capping it (:302), then transfers <code>fee * share_bps / 10000</code> of the input token out of the pool to the “referrer” (:305). A malicious registry returning a huge <code>share_bps</code> drains reserves swap by swap. No timelock.
F-06	High	Oracle: liveness	The oracle is an admin-pushed mock. On testnet its XLM price is 17.4 h stale (\$0.4), so the vault is halted: no opens, closes or liquidations. <code>close_position</code> also needs a fresh price (:551-556), which means users cannot exit positions during an oracle outage . Only free collateral can be withdrawn.

ID	Sev.	Contract / area	Finding (evidence)
F-07	Medium	Vault: staleness check	if now > pd.timestamp && now - pd.timestamp > max_price_age (:268) accepts future-dated prices forever. The mock oracle lets the admin set any timestamp (mock-oracle/src/lib.rs:51-58). PoC .
F-08	Medium	All: storage TTL	Instance TTL is never extended in staking, referral, leverage-vault or mock-oracle. extend_ttl on instance exists only in amm-pool:168-172 and reward-token:277-281. Some persistent entries are also never re-bumped: QFX Exempt(addr) (:661-663), referral Referrer (bumped only at set), oracle prices, and vault Position on set_triggers (:548). Mainnet minimum persistent TTL is about 120 days (2,073,600 ledgers). Archived entries force restore steps and fees, and break the keeper.
F-09	Critical	Keys / governance	One admin key controls all 7 contracts: GDAEZGA6...FB3VBNKCY, with one signer (weight 1) and thresholds 0/0/0 (Horizon). It was created with stellar keys generate into the CLI keystore on a dev box (scripts/deploy-testnet.sh:40). There is no multisig, no timelock and no pause in any contract. Staking, referral, vault, oracle and pool have no set_admin , so the admin can only be rotated by changing signers on the admin account.
F-10	Medium	Frontend: slippage	Pool deposit/withdraw are sent with min_a = min_b = 0 (frontend/src/pages/Pools.tsx:107,118), which is sandwichable. Swap minOut comes from a client-side quote that silently falls back to hard-coded DEMO_POOLS[0] reserves if the live read fails (Trade.tsx:69-78).
F-11	Medium	Frontend: signing UX	In-app keys sign any XDR immediately (LocalKeySigner.signTransaction, frontend/src/lib/signer.ts:40-45). invokeContract goes straight from prepare to sign to send (frontend/src/lib/soroban.ts:34-57). There is no human-readable preview of the calls, auth entries or balance changes.
F-12	Medium	Legacy QFX	The retired testnet QFX CDUMMYBM...E774HZ still exports an admin-gated mint (source git show 782f9ad^:contracts/reward-token/src/lib.rs line 380; confirmed in the on-chain interface) and has 5,000,000 unbacked supply . The new QFX has no mint (on-chain interface and source). The risk is accidental reuse of legacy wasm, IDs or aliases in a mainnet script.
F-13	High	QFX: liability	The APR is fixed (12% deployed, up to 25% MAX_APR_BPS, :55) and applies to unbounded eligible supply. The reserve is finite and cannot be withdrawn (there is no function for it). When the reserve is empty, yield silently stops, so the advertised APR is not honored. A 90-day run needs 3.00% of eligible supply (\$10).
F-14	High	Vault: toxic flow	Opens and closes execute at the last oracle price. Reflector samples every 5 minutes, and the testnet config accepts prices up to 15 minutes old. There is no spread, price-impact fee, execution delay, minimum hold, open-interest (OI) cap, skew limit, funding fee or borrow fee, so informed traders can arbitrage stale prices against the reserve.
F-15	Medium	Product claims	The landing-page leverage slider goes to 20x (frontend/src/pages/Landing.tsx:471) and the outline says "up to 20 times" (docs/landing-outline.md:45). The deployed vault caps at 10x (scripts/deploy-testnet.sh:90; live config). The contract hard cap is 20x (leverage-vault/src/lib.rs:37). USDC/EURC get a "✓ Allowlisted issuer" badge (Landing.tsx:258) even though the issuer's stellar.toml was not verified.
F-16	Medium	Web hosting	There is no CSP meta tag (frontend/index.html), and GitHub Pages cannot set response headers (live curl -I shows HSTS but no CSP or X-Frame-Options). The origin https://yogibear1323.github.io is shared by every Pages site on that GitHub account , so any of them can read localStorage["quasaria.keystore.v1"] (the encrypted key blob) and brute-force it offline.

ID	Sev.	Contract / area	Finding (evidence)
F-17	Low	AMM	swap_prepaid takes an unauthenticated trader argument used for referral attribution (amm-pool/src/lib.rs:536-547). Combined with sybil referrers, this lets a trader rebate up to share_bps of their own fee. Anyone can call sync (:551-557) to donate and move the price, so pool prices must never be used as an oracle. Referral tokens are transferred out before reserves are written (:305 vs :318), which is not checks-effects-interactions (CEI) order.
F-18	Low	Staking: admin inputs	add_pool and set_reward_rate accept a negative reward_rate (staking/src/lib.rs:175-208). There is no withdraw, so unused reward reserve is stranded after deactivation. Rounding dust is stranded too.
F-19	Low	QFX: constructor	The constructor only checks that the collateral token has 7 decimals (reward-token/src/lib.rs:508-511), not that it is the native XLM SAC. deploy-testnet.sh doesn't check it either; only migrate-qfx-peg-testnet.sh:52 does.
F-20	Medium	Keeper	The keeper is a single process running an infinite loop that only logs to the console (bot/src/index.ts:84-87, 95-108). In strategy mode, keeper errors are swallowed (:106, .catch(() => void 0)). It uses a fixed BASE_FEE inclusion fee with no surge bump or retry (bot/src/soroban.ts:50) and runs O(N) sequential simulations per sweep. There is no health endpoint and no alerting. QUASARIA_SECRET sits in plaintext in .env.
F-21	Low	AMM LP-share token	The LP token's approve (amm-pool/src/lib.rs:566-586), burn and burn_from (:603-611) emit no SEP-41 events . transfer/transfer_from do. Indexers, wallets and explorers will mis-track LP allowances and supply. (Scout "Token Interface should emit an event"; confirmed by reading the code.)

1. Smart-contract security

1.1 External audit by a recognized Soroban auditor

- **Covers:** An independent review of all 7 contracts, plus any replacements (a Reflector adapter instead of the mock oracle), by a firm with a Soroban track record. Examples: firms on SDF's Soroban Audit Bank list, such as Certora, OtterSec, Runtime Verification and Veridise, or the other firms in the repo's docs/audit-firm-shortlist.md.
- **Pass looks like:** The final commit hash is audited. All Critical/High findings are fixed and verified in a re-audit. Medium findings are fixed or formally accepted with reasons. The report is published, and the deployed wasm hashes match the audited commit.
- **Evidence that should exist:** audits/<firm>-<date>.pdf, a remediation log, and the re-audit letter, with the audited commit and wasm hashes listed.
- **Current status: Not started.** There is no audits/ directory and no report. Every contract carries the meta tag "testnet-only scaffold, unaudited" (e.g., contracts/reward-token/src/lib.rs:50). This review found 2 Critical and 7 High issues (\$0.6) that must be fixed before an audit is worthwhile.
- **Blocks mainnet: YES**

1.2 Internal security review and threat model

- **Covers:** A documented internal line-by-line review, a STRIDE threat model (the SDF Audit Bank requires one), a trust-assumption list (admin, oracle, SAC, keeper), and invariants per contract.

- **Pass looks like:** `docs/security/threat-model.md` exists with assets, actors, trust boundaries and mitigations. Each invariant maps to a test. Findings are tracked to closure.
- **Evidence that should exist:** The threat-model document, an invariant list, and closed issues or PRs for each finding.
- **Current status: Partial.** This checklist is an internal review: it found 21 findings (§0.6) and 5 PoCs (§0.5). There is still no threat model, invariant specification or finding tracker in the repo. README security notes (e.g., lines 3, 23, 741) are warnings, not a model.
- **Blocks mainnet: YES**

1.3 Static analysis (clippy, cargo-audit, Scout)

- **Covers:** Lints for correctness and arithmetic, advisories on the dependency tree, and Soroban-specific detectors (CoinFabrik Scout).
- **Pass looks like:** Clippy runs clean with `-D warnings` plus `clippy::arithmetic_side_effects`, `unwrap_used` and `cast_possible_truncation` (or each justified allow). `cargo audit` shows 0 vulnerabilities. The Scout report has no unresolved Critical/Medium issues. All of this runs in CI.
- **Evidence that should exist:** CI logs and archived reports (`reports/clippy.txt`, `reports/cargo-audit.json`, `reports/scout.md`).
- **Current status: Partial.** Clippy (default) gives 11 style warnings and 0 correctness lints. Clippy (strict) gives 113 `arithmetic_side_effects`, 30 `integer_division`, 24 `unwrap_used` and 3 truncating casts (`vault :289, :462; token :727`). `cargo audit` shows 0 vulnerabilities and 1 “unmaintained” warning (paste RUSTSEC-2024-0436, pulled in transitively by soroban-sdk). Scout (v0.3.17, detectors nightly-2025-09-18): 111 Critical (all unchecked arithmetic, overlapping clippy), 51 Medium, 42 Enhancement. The true positives are F-02 (dynamic storage), the single-step `set_admin`, and missing LP-token events (F-21). Details in §0.2. No CI runs any of these (§8.9).
- **Blocks mainnet: NO** (the findings it produces feed 1.6)

1.4 Fuzz and property/invariant tests

- **Covers:** `cargo fuzz` or `proptest` harnesses plus Soroban `testutils` invariant tests. Invariants: `QFX supply == collateral held` (`assert_backed`, `reward-token/src/lib.rs:329-334`), `AMM k non-decreasing`, `staking sum(claimable) <= reserve`, `vault liquidity + margins == token balance`, and liquidation always possible when `HF < 1`.
- **Pass looks like:** Each contract has invariant tests over random operation sequences, including extreme values (1 stroop, i128 bounds, zero supply). The harnesses run in CI.
- **Evidence that should exist:** `contracts/*/fuzz/` or `proptest` modules, with corpus and run logs.
- **Current status: Not started.** `grep -r "proptest\|fuzz_target\|arbitrary" over contracts/` returns nothing. Findings F-02 and F-04 are exactly the edge cases fuzzing catches.
- **Blocks mainnet: YES**

1.5 Test coverage

- **Covers:** Line and branch coverage of the production `lib.rs` files, including negative/auth tests.
- **Pass looks like:** $\geq 95\%$ line coverage and $\geq 90\%$ branch coverage on production code. Every admin function has a positive test and an unauthorized-caller test.
- **Evidence that should exist:** A `cargo llvm-cov` report in CI, and a coverage badge.

- **Current status: Partial.** 41/41 tests pass. Line coverage is **88.8%** (1,582/1,781): AMM 72.3%, oracle 79.6%, router 91.9%, vault 92.6%, staking 95.4%, token 95.7%, referral 96.4%. Untested: AMM `sync` (:551-557), LP-share `approve/transfer_from/burn` (:563-617), `set_fee_bps/set_referral`, the deposit ratio branch (:454-458); vault `set_oracle` (:367-370); QFX `set_admin` (:630-634).
- **Blocks mainnet:** NO (should reach target before the audit freeze)

1.6 Arithmetic overflow and rounding

- **Covers:** Overflow in i128 multiplications, precision loss, rounding direction (always in the protocol's favor), and division by zero.
- **Pass looks like:** Checked math (or `overflow-checks` with proven bounds) everywhere, with fixed-point scaling chosen so realistic values can't overflow. Minimum amounts stop dust from rounding fees or maintenance to zero. There is a rounding-direction table per formula.
- **Evidence that should exist:** An arithmetic spec, bounds proofs or property tests, and minimum-size constants.
- **Current status: Fail.** `overflow-checks = true` in the release profile, so overflow panics instead of wrapping (safe, but it causes DoS). Confirmed bugs: **F-04**, where the staking accumulator overflows and bricks the pool (`staking/src/lib.rs:153,161,256`), and **F-02**, where the vault fee and maintenance margin round to 0 for 1-stroop margin (`leverage-vault/src/lib.rs:475-476,179-182`). There are 3 truncating casts. QFX yield rounding favors the reserve (good). AMM math uses `mul_div`-style floor rounding that favors the pool.
- **Blocks mainnet: YES**

1.7 Reentrancy and cross-contract call safety

- **Covers:** Soroban blocks direct re-entry into the same contract. Cross-contract callbacks and trust in external contracts (SAC tokens, referral registry, oracle) remain risks.
- **Pass looks like:** CEI order in every function that transfers tokens. Each external contract address is fixed at construction, or changed only through a timelock. Return values from external contracts are bounds-checked (e.g., `share_bps <= MAX`). Only allowlisted SAC tokens are used.
- **Evidence that should exist:** A call graph, a trust table, and tests with malicious mock contracts.
- **Current status: Partial.** Router → pool calls use fixed pool addresses, and user state is written before transfers in most places. However, the AMM trusts the registry's `share_bps` without re-capping it (**F-05**, `amm-pool/src/lib.rs:302`) and pays the referral before writing reserves (:305 vs :318). The vault trusts whatever oracle the admin sets (**F-01**). No tests use hostile mock contracts.
- **Blocks mainnet: YES**

1.8 Authorization (`require_auth`) on every state-changing function

- **Covers:** Every function that moves user funds or changes user state authenticates the owner. Admin functions authenticate the admin.
- **Pass looks like:** Each function is authenticated or is explicitly permissionless by design, with a documented reason.
- **Evidence that should exist:** A function/auth matrix (see §5.1) and unauthorized-caller tests.
- **Current status: Pass.** Every user-state function reviewed calls `require_auth` on the owner: QFX `deposit/redeem/transfer/burn` (burn redeems 1:1, `reward-token/src/lib.rs:813-823`), AMM

`deposit/withdraw/swap`, router `swap_exact_in`, staking `stake/unstake/claim`, referral `set_referrer` (`referral/src/lib.rs:133`), vault `deposit/withdraw/set_operator` (owner only), and `open_position/close_position/set_triggers` (owner or delegated operator, via `require_controller`). Every admin function calls `admin.require_auth()`. Functions that are permissionless by design: AMM `sync` and `swap_prepaid` (the router pays first), vault `execute_trigger` (keeper), QFX `accrue/settle/sweep_surplus` (a donated surplus becomes yield, :603-620), and the funding entry points `fund_liquidity/fund_yield/staking_fund`, which authenticate the payer. Vault `liquidate` authenticates the liquidator (:577). One caveat is the unauthenticated trader in `swap_prepaid` (**F-17**, low).

- **Blocks mainnet:** NO

1.9 Storage TTL and archival handling

- **Covers:** Instance, persistent and temporary storage TTL bumps; restore paths for archived entries; keeper and frontend handling of archived-entry errors.
- **Pass looks like:** Every contract extends instance TTL on each call (or on a schedule run by the keeper). Every persistent entry is bumped when read or written. There is a TTL runbook and an automated “bump” job, and archived-entry restore has been tested.
- **Evidence that should exist:** Code that extends TTL, a TTL monitoring job, and restore tests.
- **Current status: Fail.** See **F-08**: instance TTL is never extended in 4 of the 7 contracts, and several persistent entries are never re-bumped. The mainnet minimum persistent TTL is about 120 days (Stellar storage-strategies docs).
- **Blocks mainnet:** YES

1.10 Unbounded storage, loops and DoS

- **Covers:** Growth of vectors and maps in single entries, per-tx resource limits (65,536 B per entry, 400M instructions, 132,096 B written, 200 entries), and griefing with dust.
- **Pass looks like:** No unbounded collections in one entry. Per-user keys, or paginated indexes. Minimum sizes for positions and stakes. Resource usage is benchmarked at 10× the expected scale.
- **Evidence that should exist:** Resource benchmarks and minimum-size constants.
- **Current status: Fail.** **F-02**: the vault’s global `openIds` hits the entry limit at about 5,460 positions, and dust positions can’t be liquidated. The router’s `MAX_HOPS = 4` is bounded (fine). The staking pool list is admin-controlled (fine).
- **Blocks mainnet:** YES

1.11 Upgradeability and migration policy

- **Covers:** Whether contracts can be upgraded (`update_current_contract_wasm`), who can do it, the timelock, and how state migrates, or, if immutable, how users migrate.
- **Pass looks like:** A deliberate, documented choice. Either immutable with a published migration and pause plan, or upgradeable behind multisig plus a timelock, with upgrade events and an audited upgrade path.
- **Evidence that should exist:** `docs/upgrade-policy.md` and a migration runbook (the QFX v1 to v2 migration is a partial precedent).
- **Current status: Not started.** All contracts are immutable. `grep update_current_contract_wasm` finds nothing. There is no pause either (**F-13**, §5.5), so a bug found after launch can only be handled by redeploying and asking users to move. `scripts/migrate-qfx-peg-testnet.sh` shows a testnet migration, but there is no policy.

- **Blocks mainnet: YES**

1.12 Bug bounty and security contact

- **Covers:** A public bounty (e.g., ImmuneFi or HackerOne), `SECURITY.md`, a disclosure email or PGP key, and a safe-harbor policy.
- **Pass looks like:** The bounty is live before TVL ramps. Critical reward is sized relative to TVL (common practice is about 10% of funds at risk, capped). `SECURITY.md` has a contact and SLA.
- **Evidence that should exist:** The bounty page URL, `SECURITY.md`, and a funded bounty wallet.
- **Current status: Not started.** There is no `SECURITY.md` in the repo and no bounty. Public reference points: Stellar's HackerOne programme (critical = 10% of funds affected, capped) and OpenZeppelin Stellar on ImmuneFi (10% of funds affected, min \$5k, max \$25k).
- **Blocks mainnet: YES**

2. Economic and tokenomics review

2.1 QFX peg invariant (1 QFX = 1 XLM, fully backed)

- **Covers:** Mint only happens on XLM deposit, burn/redeem is 1:1, `total_supply` $\leq$ XLM held, and yield is paid from pre-funded XLM, not new issuance.
- **Pass looks like:** The invariant is enforced on-chain after every state change and fuzzed. No admin path creates unbacked QFX. The collateral is provably the native SAC. There is a public proof-of-reserves view.
- **Evidence that should exist:** An invariant test/fuzz suite, a published SAC address check, and a reserves dashboard.
- **Current status: Partial.** The design is sound. `deposit` (`reward-token/src/lib.rs:533-549`) mints only against an XLM transfer, and `redeem/burn` (`:553, :813-823`) return XLM 1:1. `assert_backed` (`:329-334`) runs after state changes. `fund_yield` (`:561`) and `sweep_surplus` (`:603-620`) keep yield backed. The admin has **no** mint (the section header at `:624` reads "admin (cannot create QFX)"). The live read shows 8,500 supply / 8,500 backing plus a 2,500 reserve. Gaps: no fuzzing (§1.4), and the constructor doesn't pin the native SAC (**F-19**).
- **Blocks mainnet: NO** (on its own; fuzzing is covered in 1.4)

2.2 Holder-yield reserve sustainability

- **Covers:** Whether the advertised APR can be paid over time, how the reserve depletes, what happens when it is empty, and yield-timing gaming.
- **Pass looks like:** A deposit or eligible-supply cap sized to the funded reserve (§10.1). Yield is time-weighted, or the minimum hold is at least 1 day. UI shows "reserve runway" and "yield stops when the reserve is empty". The admin can lower the APR only via timelock.
- **Evidence that should exist:** A reserve-runway model, a cap parameter, and a fixed accrual design with tests.
- **Current status: Fail. F-13:** unbounded liability against a finite, non-withdrawable reserve. **F-03:** boundary sniping (PoC: 328.77 XLM taken from a 120-second deposit of 1M). On the live testnet, the 2,500 XLM reserve covers only about 83k eligible QFX for 90 days (§10.1). `set_yield_exempt` (`:651`) lets the admin silently switch off any holder's yield (a centralization and disclosure issue).
- **Blocks mainnet: YES**

2.3 Staking reward runway

- **Covers:** Fixed per-second emission per pool, reserve balance, runway, APR at different TVLs, and the behavior when the reserve runs out.
- **Pass looks like:** Runway ≥ 90 days at launch with alerts at 30 days. The UI shows APR computed from live TVL, not a static figure. Rewards are capped at the reserve without reverting.
- **Evidence that should exist:** A runway dashboard, top-up procedure, and `pending_rewards <= reserve` tests.
- **Current status: Partial.** Live reads: pool 0 has a 1,498.39 QFX reserve at 8.64 QFX/day, which is **173 days** of runway. Pool 1 has 1,496.79 QFX at 17.28 QFX/day, which is **86.6 days**, just under 90. There is no alerting, and the rate can be set negative (**F-18**). The accumulator overflow **F-04** is tracked under §1.6.
- **Blocks mainnet:** NO

2.4 Referral fee split

- **Covers:** `share_bps` of the swap/open fee paid to referrers, caps, sybil/self-referral, and attribution.
- **Pass looks like:** The cap is enforced **where the fee is paid** (in the pool and vault), not only in the registry. Self-referral is prevented or accepted as a documented rebate. Attribution is authenticated.
- **Evidence that should exist:** Tests covering a malicious registry, self-referral policy, and emitted events.
- **Current status: Partial.** The registry caps `share_bps` at `MAX_SHARE_BPS = 5000` (`referral/src/lib.rs`, `set_share_bps :104`), and 2,000 (20% of the fee) is deployed (`scripts/deploy-testnet.sh:70`). `record_reward` requires source auth and an allowlist (`:194-195`). However, the pool does **not** re-cap (**F-05**), and attribution can be spoofed via `swap_prepaid` (**F-17**).
- **Blocks mainnet:** NO (the drain aspect is a blocker under 5.6)

2.5 AMM fee and constant-product invariant

- **Covers:** The $x \cdot y = k$ math, fee accounting, the effect of the referral carve-out on k , LP share mint/burn rounding, and donation/`sync` behavior.
- **Pass looks like:** k never decreases across swaps (property-tested). LP share rounding favors the pool. Fee bounds are enforced. `sync` donations can't hurt LPs.
- **Evidence that should exist:** Property tests and a written spec of the fee path.
- **Current status: Partial.** The fee is 30 bps (`deploy-testnet.sh`), with `MAX_FEE_BPS = 100` (`amm-pool/src/lib.rs:25`). The referral cut is taken out of the input amount before reserves are updated (`:302-313`), so LPs receive $\text{fee} \times (1 - \text{share})$. With a 20% share, LPs keep 24 bps and referrers get 6 bps. Floor rounding favors the pool. There is no k -invariant property test, and 72.3% line coverage is the lowest of any contract.
- **Blocks mainnet:** NO

2.6 Manipulation vectors (flash-style swaps, oracle and pool price manipulation)

- **Covers:** Same-transaction multi-call attacks (Soroban has no native flash loans, but a single tx can chain calls), sandwiching, using pool prices as oracles, oracle-latency arbitrage, and

donation attacks.

- **Pass looks like:** No contract reads spot pool prices as an oracle. The vault has an execution delay, a spread or a minimum hold, and blocks same-ledger open/close. Every user flow has slippage bounds. Oracle deviation checks are in place (§3.3).
- **Evidence that should exist:** A manipulation analysis document, simulations, and tests.
- **Current status: Fail.** Pool prices are not used as oracles (good). However: **F-14** (oracle-latency toxic flow, no delay or spread, open and close allowed in the same tx), **F-10** (UI sends $\text{min} = 0$ on liquidity operations), **F-17** (sync donations and spoofed attribution), and **F-07** (future-dated prices accepted).
- **Blocks mainnet: YES**

2.7 Leverage-vault liquidation math

- **Covers:** Health factor, maintenance margin, liquidation price, bonus, rounding, and whether liquidation is always possible and incentivized.
- **Pass looks like:** HF is monotonic in price. Every position above a minimum size has $\text{maint} > 0$. The liquidation bonus covers gas even when underwater (or the protocol pays keepers). The positions and liquidations are tested at the extremes.
- **Evidence that should exist:** A math spec, property tests, and a keeper-incentive analysis.
- **Current status: Partial.** The formulas are correct for normal sizes: $\text{pnl} = \text{size} \cdot \Delta p / \text{entry}$ (leverage-vault/src/lib.rs:168-171), $\text{HF} = \text{equity} / (\text{size} \cdot \text{mm})$ (:174-184), and liquidation price (:187-196). With 10× leverage and a 5% maintenance margin, a position is liquidated after about a 5% adverse move (deploy-testnet.sh:90). `validate_config` (:288-304) forces maintenance below initial margin. Defects: **F-02** (maint rounds to 0, so $\text{HF} = \text{MAX}$ and dust can't be liquidated). The bonus is $\text{min}(\text{equity}, 5\% \text{ of margin})$ (:590-591), so it is **0 for underwater positions** and nobody is paid to liquidate them. Liquidation needs a fresh oracle price (**F-06**).
- **Blocks mainnet: YES**

2.8 Bad debt, solvency and open-interest limits

- **Covers:** Who absorbs trader profit, reserve solvency under large moves, OI caps, per-position caps, skew limits, funding/borrow fees, and auto-deleveraging.
- **Pass looks like:** $\text{Net OI} \leq \text{reserve} \times \text{safety factor}$. Per-account and per-market caps. Funding or borrow fees balance skew. When the reserve can't pay, the rule is explicit and disclosed (pro-rata, ADL), not silent truncation.
- **Evidence that should exist:** A risk-parameter document, a stress model and on-chain caps.
- **Current status: Fail.** The vault is the sole counterparty. Losses are capped at margin, so there is no negative-equity debt, but **profits are silently cut to the remaining liquidity** (`settle`, :307-321). There are no OI, size or skew caps and no funding/borrow fees (**F-11**, **F-14**). The admin can withdraw the reserve at any time (**F-01**).
- **Blocks mainnet: YES**

2.9 Insurance / backstop

- **Covers:** An insurance fund, a cover provider, or a treasury backstop for exploits and vault shortfall.
- **Pass looks like:** A decision is documented: fund size, trigger rules and custody, or explicit "no insurance" disclosures.
- **Evidence that should exist:** A treasury policy and disclosure text.

- **Current status: Not started.** Nothing in the repo.
- **Blocks mainnet:** NO (the disclosure is required under 9.6)

2.10 Economic stress testing

- **Covers:** Simulating XLM $\pm 40\text{--}60\%$ moves, a bank run on QFX, reserve depletion, and a liquidation cascade with a slow keeper.
- **Pass looks like:** A reproducible simulation notebook with parameters chosen from its results.
- **Evidence that should exist:** `research/stress/*.ipynb` or a script, plus a report.
- **Current status: Not started.** The bot's strategy engine and risk module (`bot/src/engine.ts`, `bot/src/risk.ts`) cover trading strategies, not protocol solvency.
- **Blocks mainnet: YES**

3. Oracle

3.1 Replace the mock oracle with a production oracle

- **Covers:** Using Reflector (for example the external CEX/DEX "Pulse" feed `CAFJZQWSED6YAWZU3GWRTOCNPPCGBN32L7QV43XX5LZLFTK6JLN34DLN`: USD base, 14 decimals, 5-minute resolution, 4-of-7 multisig operated; or the Stellar-DEX feed `CALI2BYU...PLE6M`, USDC base) or another production Stellar oracle, with an audited adapter.
- **Pass looks like:** The vault reads a production feed through a SEP-40 (`lastprice`) client. The oracle address is fixed at construction or behind a timelock. The mock oracle wasm isn't deployed to mainnet at all.
- **Evidence that should exist:** Adapter code and tests against the Reflector interface, a mainnet config, and an upkeep/subscription arrangement.
- **Current status: Fail.** Testnet uses `mock-oracle` (`contracts/mock-oracle/src/lib.rs`), which the admin pushes with any price and timestamp (`:51-66`). It is 17.4 h stale on-chain (**F-06**). The vault's oracle client is SEP-40-shaped (`lastprice`), so swapping it in is feasible, but `set_oracle` has no timelock (**F-01**).
- **Blocks mainnet: YES**

3.2 Staleness checks

- **Covers:** Rejecting prices older than N seconds and prices from the future. N must match the feed's resolution.
- **Pass looks like:** `max_age` $\approx 1\text{--}2 \times$ the feed resolution (5–10 minutes for Reflector). Future timestamps are rejected (beyond a small clock-skew allowance). There is a documented user exit path while prices are stale.
- **Evidence that should exist:** Tests for stale, future and boundary cases.
- **Current status: Partial.** There is a check (`oracle_price`, `leverage-vault/src/lib.rs:259-272`, `max_price_age = 900`), but future timestamps pass (**F-07**, PoC). Close/liquidate also revert when stale (**F-06**), leaving no exit.
- **Blocks mainnet: YES**

3.3 Deviation bounds / sanity checks

- **Covers:** Rejecting prices that jump more than X% from the last accepted price, cross-checking two feeds, and rejecting zero or negative prices.

- **Pass looks like:** A deviation threshold (e.g., 5–10%) that pauses the market when it trips. Two independent sources (e.g., Reflector CEX/DEX vs the Reflector Stellar-DEX or another provider) must agree within a tolerance.
- **Evidence that should exist:** Code and tests, plus parameter rationale.
- **Current status: Not started.** Only `price > 0` is checked, and there is no deviation check (`:259-272`).
- **Blocks mainnet: YES**

3.4 Fallback oracle

- **Covers:** Behavior when the primary feed is stale or disagrees: a secondary feed, TWAP, or a pause with reduce-only mode.
- **Pass looks like:** A documented and tested fallback, or reduce-only close at the last good price with a discount.
- **Evidence that should exist:** Code and a runbook.
- **Current status: Not started.**
- **Blocks mainnet: YES**

3.5 Oracle operations

- **Covers:** Reflector upkeep/subscription fees, monitoring feed freshness, asset symbol mapping (`XLM as Other(Symbol)` vs `Stellar(Address)`), and the choice of base asset (USD vs USDC).
- **Pass looks like:** A freshness alert (for example, `age > 2 × resolution`), a funded upkeep balance, and a documented asset mapping.
- **Evidence that should exist:** A monitoring dashboard and an ops document.
- **Current status: Not started.**
- **Blocks mainnet: NO**

4. Liquidity, staking and pool contracts

4.1 Initial liquidity plan

- **Covers:** Which pools launch, their per-side depth, the source of funds (treasury or partners), the price at seeding, the LP-share custody wallet, and the plan for withdrawing.
- **Pass looks like:** A written plan with amounts (see §10.3). Seeding goes through the router/pool with a nonzero min at the market price, from a multisig. LP shares are held by the treasury multisig.
- **Evidence that should exist:** `docs/launch/liquidity-plan.md` and a signed treasury approval.
- **Current status: Not started.** Only testnet seeding exists (`scripts/seed-testnet.sh`), and it uses `min 0` (`:40-41`). Live testnet pools are tiny: XLM/QUSD has 576.99 QUSD on side B, and QFX/QUSD has 277.41 QUSD.
- **Blocks mainnet: YES**

4.2 Minimum liquidity and first-depositor attack

- **Covers:** Share inflation by the first depositor, a `donation/sync` before the first deposit, and deposits at a skewed ratio.
- **Pass looks like:** Dead shares are locked on the first mint. Protocol-owned seeding happens atomically at deploy. Deposits take a `min_shares` parameter.

- **Evidence that should exist:** Tests for first deposit + donation, and a deploy script that seeds atomically.
- **Current status: Partial.** `MINIMUM_LIQUIDITY = 1000` is burned on the first deposit (`amm-pool/src/lib.rs:26, :437-445`), which is the standard Uniswap-v2 mitigation. There is no `min_shares` parameter (only `min_a/min_b`). The ratio branch (`:454-458`) is untested. Pools are deployed and seeded in separate transactions.
- **Blocks mainnet:** NO

4.3 Slippage and deadline protection (contract and UI)

- **Covers:** Router `min_out` and deadline, pool-level `min_*`, and the values the UI actually sends.
- **Pass looks like:** Every user-facing operation sends a nonzero min derived from a fresh on-chain quote and a user-set tolerance, plus a deadline. The UI refuses to quote from demo data on mainnet.
- **Evidence that should exist:** UI tests asserting the mins, and the router tests.
- **Current status: Partial.** The contracts are good: the router enforces the `deadline` (`router/src/lib.rs:103`) and `min_out` (`:124`), with `MAX_HOPS = 4` (`:32`), and the test `deadline_and_slippage_enforced` passes. The pool enforces `min_a/min_b/min_out`. The UI is not: **F-10** (`Pools.tsx:107,118` send 0, and the Trade quote falls back to `DEMO_POOLS`, `Trade.tsx:69-78`). Direct `pool.swap` has no deadline (only the router does).
- **Blocks mainnet:** YES

4.4 Withdrawal paths under stress

- **Covers:** Whether users can always exit when the oracle is down, the reserve is empty, storage is archived, the frontend is down, or the admin key is lost.
- **Pass looks like:** Every exit is permissionless and independent of the oracle and admin (or reduce-only when the oracle is down). There is a CLI/“escape hatch” document for exiting without the frontend.
- **Evidence that should exist:** Stress tests, and `docs/emergency-exit.md` with CLI commands.
- **Current status: Partial.** QFX `redeem`, AMM `withdraw`, staking `unstake` and vault free-collateral `withdraw` don’t depend on the admin or oracle (good). However: vault close needs a fresh oracle (**F-06**); vault profits depend on reserve liquidity that the admin can withdraw (**F-01**); stakers’ pools can be bricked by **F-04** (`unstake` of existing positions still works, but new stakes panic); archived storage needs restore (**F-08**). There is no escape-hatch document.
- **Blocks mainnet:** YES

4.5 Staking lifecycle

- **Covers:** Adding pools, funding, rate changes, deactivating, the claim/unstake ordering, and leftover rewards.
- **Pass looks like:** Rates validated (`> 0`, bounded). Rewards accrue correctly across rate changes. Leftover rewards can be recovered via timelock or are explicitly burned. Pools are funded before they are activated.
- **Evidence that should exist:** Lifecycle tests.
- **Current status: Partial.** `updated()` accrues before every rate or active change (`staking/src/lib.rs:205,213`). Anyone can fund (`:219-232`). The 6 tests pass. There are gaps: **F-18** (negative rate, stranded reserve) and **F-04**. `set_active(false)` stops new stakes and rewards but lets users exit (`:210-216`). This should be documented for users.
- **Blocks mainnet:** NO (F-04 is counted under 1.6)

4.6 Asset listing and issuer verification (USDC/EURC etc.)

- **Covers:** Verifying stablecoin issuers via their `stellar.toml/home` domain, the asset allowlist, and the UI badge wording.
- **Pass looks like:** Issuer accounts checked against Circle’s published `stellar.toml`, SEP-1 verified (home domain shown), SAC contract IDs pinned, and a “verified” badge shown only after verification.
- **Evidence that should exist:** An allowlist file with issuer, toml URL, date checked, and SAC ID.
- **Current status: Partial.** Landing marks USDC/EURC as “✓ Allowlisted issuer” (`Landing.tsx:258`) because Circle’s `stellar.toml` could not be verified (a known caveat). On testnet, 20 of the 21 stablecoin pools use mock tokens (`deployments/testnet-stablecoins.json`), and QUSD is admin-issued (`deploy-testnet.sh:65`).
- **Blocks mainnet: YES**

5. Access control and admin keys

5.1 Privileged-function inventory (from the code)

- **Covers:** Every function gated by an admin, what it can do in the worst case, and whether a timelock or bound applies.
- **Pass looks like:** A complete table (below) that stays up to date and is reviewed by the auditor. Every “worst case” is either removed, bounded, or behind multisig plus timelock.
- **Evidence that should exist:** This table, kept in `docs/security/privileged-functions.md`.
- **Current status: Pass** (the inventory is complete; the *risks* it shows are tracked in 5.2–5.6).
- **Blocks mainnet: NO**

Contract (testnet id in <code>deployments/testnet.json</code>)	Privileged function (file:line)	Bound in code	Worst case if the admin key is compromised	Admin rotation
QFX reward-token	<code>set_admin</code> (:630-634)	none; single-step, no event	Hand control to an attacker; the new address can’t be sanity-checked	yes (single-step)
	<code>set_apr_bps</code> (:638)	$\leq$ <code>MAX_APR_BPS</code> 2,500 (:55)	APR 0 (yield stops) or 25% (reserve drains 2× faster)	
	<code>set_yield_exempt</code> (:651)	none	Silently turn off any holder’s yield	
	<i>(no mint; no reserve withdraw)</i>	—	Cannot create QFX or take XLM backing ✓	
AMM amm-pool (×2 on testnet + stablecoin pools)	<code>set_fee_bps</code> (:364-370)	$\leq$ <code>MAX_FEE_BPS</code> 100 (:25)	Fee up to 1%	none: admin fixed at construction
	<code>set_referral</code> (:372-378)	none	Point at a malicious registry and drain reserves (F-05)	
Router <code>router</code>	— (no admin)	—	—	n/a

Contract (testnet id in deployments/testnet.json)	Privileged function (file:line)	Bound in code	Worst case if the admin key is compromised	Admin rotation
Staking staking	add_pool (:175-201)	lock ≤ MAX_LOCK_SECONDS	Add pools with arbitrary tokens/rates (negative allowed, F-18)	none
	set_reward_rate (:203-208)	none	Rate 0 (stop rewards) or huge (exhaust the reserve early); negative	
	set_active (:210-216)	—	Block new stakes and stop rewards (users can still exit)	
Referral referral	set_share_bps (:104-111)	≤ MAX_SHARE_BPS 5,000 (:29)	Up to 50% of fees to referrers	none
	set_fee_source (:113-122)	none	Allow a rogue contract to record fake earnings (bookkeeping only)	
Leverage vault leverage-vault	set_config (:361-365)	validate_config (:288-304): leverage ≤ 20× hard cap, maint < initial margin, bonus ≤ 20%, fee ≤ 1%	Raise leverage to 20×, change maintenance margin mid-flight (instantly liquidating users)	none
	set_oracle (:367-370)	none	Point at an attacker oracle, mark all positions liquidatable or pay fake profits (F-01)	
	set_market (:372-384)	—	Disable a market (blocks new opens only; :471)	
	withdraw_liquidity (:396-405)	≤ current liquidity	Take the whole counterparty reserve , including third-party fund_liquidity deposits (F-01)	
Mock oracle mock-oracle	set_price (:51-66)	none (any price, any timestamp)	Arbitrary prices; must never go to mainnet	none

Admin on every contract: `GDAEZGA66NUQZKMHFMIUI42S6FKGFERY3UHAN43ZM3RGL3FB3VBNKCY`, the same key for all of them (deployments/testnet.json; scripts/deploy-testnet.sh:39-41).

5.2 Key custody (who holds the keys, hardware wallets)

- **Covers:** Where admin, treasury, operator and keeper keys live, who holds them, and backup/recovery.
- **Pass looks like:** Admin and treasury keys are hardware-wallet signers (Ledger via Freighter, the Stellar CLI's Ledger support, or a similar wallet) held by named, independent people in different locations, with documented backups (SLIP-39 or metal seed storage) and an annual recovery drill. No privileged key ever touches a dev laptop keystore or CI.
- **Evidence that should exist:** A key ceremony record, a custodian list (internal) and a recovery drill log.
- **Current status: Fail.** One hot key for everything, generated with `stellar keys generate` into the CLI keystore (`deploy-testnet.sh:40`). Horizon shows a single signer and thresholds 0/0/0 (**F-09**). Scripts write deploy outputs to `.env` files.
- **Blocks mainnet: YES**

5.3 Multisig plan (Stellar account thresholds)

- **Covers:** Making each admin address a Stellar multisig account. Soroban `require_auth` on a G-account is satisfied by signatures meeting the account's **medium** threshold (Stellar authorization docs), so the account's thresholds *are* the contract's multisig.
- **Pass looks like:** Admin account: master weight 0, 3-of-5 (or 4-of-7) hardware signers of weight 1, med = high = 3 (or 4), low ≥ 1 . There are separate accounts for each role: protocol admin, treasury, oracle/market operations, pause guardian (lower threshold, e.g. 2-of-5, pause-only). Thresholds are tested on testnet by signing an admin call with M-1 signers (must fail) and with M signers (must pass). Because staking, referral, vault, oracle and AMM have **no** `set_admin`, the admin must be the multisig account **at deploy time**. Rotation then happens by changing signers on that account.
- **Evidence that should exist:** A signer table, `set_options` transaction hashes, and a testnet rehearsal log.
- **Current status: Not started** (see F-09). Reference: developers.stellar.org, "Signatures and multisig".
- **Blocks mainnet: YES**

5.4 Timelocks

- **Covers:** A delay between announcing and executing sensitive admin actions (oracle change, referral change, fee/config/APR changes, liquidity withdrawal, upgrades).
- **Pass looks like:** An on-chain queue/execute timelock (e.g., 48-72 h), and cancellation by the guardian. Events are emitted so watchers and users can exit.
- **Evidence that should exist:** Timelock code and tests, plus monitoring of queued actions.
- **Current status: Not started.** No contract has a timelock (grep: no `eta/queue/timelock`) (**F-13**).
- **Blocks mainnet: YES**

5.5 Emergency pause / guardian

- **Covers:** The ability to halt risky entry points (deposits, opens, swaps) while leaving exits open, and who can pause and unpauses.
- **Pass looks like:** A `paused` flag per contract (or per market), set by a guardian multisig with a lower threshold. Unpause requires the admin multisig. Exits are never paused (or only via reduce-only). Tested.
- **Evidence that should exist:** Code, tests, a runbook (§8.6) and a drill log.
- **Current status: Not started.** grep finds no `pause` in any contract (**F-13**). The only partial levers are staking `set_active` and vault `set_market(false)`.
- **Blocks mainnet: YES**

5.6 Rug vectors and free-mint paths

- **Covers:** Any admin path that can mint unbacked tokens or move user or LP funds. Confirming that the retired testnet QFX's admin mint does not exist in the new contracts.
- **Pass looks like:** No admin function can mint or move user funds. Treasury-only withdrawals are bounded and timelocked. The legacy wasm hash and IDs are blocklisted in the deploy tooling.
- **Evidence that should exist:** This table, the auditor's confirmation, and a deploy-script hash allowlist.

- **Current status: Partial.** ✓ The new QFX has **no mint**. Source and the on-chain interface both confirm it (§0.3, **F-12**). ✓ No active contract has an admin-callable mint (grep). The only mint is the internal LP-share `mint_lp` (`amm-pool/src/lib.rs:189`), which is called on deposit. ✗ The legacy testnet QFX `CDUMMY...E774HZ` still exports admin `mint` (5,000,000 unbacked). ✗ The vault admin can withdraw the whole reserve and swap the oracle (**F-01**). ✗ The AMM admin can drain pools through `set_referral` (**F-05**).
- **Blocks mainnet: YES**

5.7 Admin rotation

- **Covers:** How to change the admin if a key is lost or compromised.
- **Pass looks like:** Two-step `propose/accept` with events on every contract, or a deliberate “admin = multisig account, rotate its signers” design, documented.
- **Evidence that should exist:** Code and tests, plus a runbook.
- **Current status: Partial.** Only QFX has `set_admin`, and it is single-step with no event (:630-634). The other five admin-bearing contracts can’t rotate their admin on-chain. That is acceptable **only** if the admin is a multisig account (5.3).
- **Blocks mainnet: NO** (it is satisfied by 5.3)

5.8 Role separation

- **Covers:** Separate keys for protocol admin, treasury, oracle push, keeper, bot operator and deployer.
- **Pass looks like:** Least privilege. The keeper/operator can’t withdraw funds. The deployer key has no residual powers after deploy.
- **Evidence that should exist:** A role table and addresses.
- **Current status: Partial.** The vault’s `operator` design is good: a user-delegated bot key can trade but **not withdraw** (`leverage-vault/src/lib.rs:17, :419, :432-446`). The keeper `liquidate` is permissionless. Everything else is one key (deployer = admin = oracle pusher = QUSD issuer).
- **Blocks mainnet: NO**

6. Frontend and wallet integration

6.1 Key-generation entropy

- **Covers:** The randomness source for browser-generated Ed25519 keys.
- **Pass looks like:** CSPRNG only (`crypto.getRandomValues`). Refuses to generate without it. No custom entropy mixing.
- **Evidence that should exist:** Code review and a unit test.
- **Current status: Pass.** `generateKey` uses stellar-sdk `Keypair.random()` (`tweetnacl randomBytes` → `crypto.getRandomValues`) and throws when Web Crypto is missing (`frontend/src/lib/keys.ts:11-24`). There is a backup check that makes the user re-type a random 6-character slice (:38-56).
- **Blocks mainnet: NO**

6.2 Remember-me key storage and encryption

- **Covers:** The encrypted keystore in `localStorage`, KDF strength, password policy, auto-lock, and origin isolation.

- **Pass looks like:** A memory-hard or high-iteration KDF, a minimum password strength of about 12 characters or a zxcvbn score, an idle auto-lock, a minimum iteration count enforced on decrypt, and a **dedicated origin** (custom domain). Hardware wallet or Freighter is recommended for material balances.
- **Evidence that should exist:** Code, tests, a UX warning, and a pen-test note.
- **Current status: Partial.** Good: off by default; PBKDF2-SHA256 with 600,000 iterations → AES-256-GCM with the public key as AAD (`keys.ts:58-110`); clear UI disclosure (`components/AccountModal.tsx:39`). Gaps: `MIN_PASSWORD = 8` (:71); no auto-lock; `decryptSecret` trusts `blob.iterations` with no floor (:94); and the blob sits on the **shared** `yogibear1323.github.io` **origin**, where any other Pages site on that account can read it for offline guessing (**F-16**).
- **Blocks mainnet: YES** (the origin isolation part, via 6.9)

6.3 CSP, XSS and clickjacking

- **Covers:** Content-Security-Policy, `frame-ancestors`, Trusted Types, `dangerouslySetInnerHTML` usage, and third-party scripts.
- **Pass looks like:** A strict CSP (`default-src 'self'`; `connect-src` limited to the RPC, Horizon and the market-data host; `frame-ancestors 'none'`; no `unsafe-inline`), served as a **header** by a host that supports it. No third-party script tags.
- **Evidence that should exist:** A header scan (e.g. securityheaders.com or Mozilla Observatory) and code grep results.
- **Current status: Fail.** There is no CSP (`frontend/index.html` has no `http-equiv` meta). Live headers from GitHub Pages: HSTS ✓ and HTTP→301 ✓, but **no CSP, no X-Frame-Options**, and `Access-Control-Allow-Origin: *` (§0.4). GitHub Pages can't set custom headers (**F-16**). React escapes output by default, and no third-party scripts were found in `index.html`.
- **Blocks mainnet: YES**

6.4 Dependency audit (npm audit)

- **Covers:** Known CVEs in the frontend and bot dependency trees.
- **Pass looks like:** 0 high/critical findings, re-run in CI on every PR, and lockfiles committed.
- **Evidence that should exist:** `npm audit --json` reports archived per release.
- **Current status: Pass** (as of Sep 26, 2026). `frontend`: **0 vulnerabilities** (125 deps). `bot`: **0 vulnerabilities** (132 deps). Both have `package-lock.json` committed. Tests: frontend vitest 79/79, bot vitest 20/20, bot `tsc --noEmit` clean.
- **Blocks mainnet: NO** (must stay green in CI)

6.5 Supply-chain risk

- **Covers:** Lockfiles, `npm ci`, pinned CI actions, build provenance, dependency count, and install scripts.
- **Pass looks like:** CI uses `npm ci` and actions pinned to **commit SHAs**. Renovate/Dependabot with review. SRI/provenance on releases. A minimal dependency set.
- **Evidence that should exist:** The workflow files and Dependabot config.
- **Current status: Partial.** Lockfiles are committed. The Pages workflow uses tag-pinned actions (`actions/checkout@v7`, `setup-node@v7`, `upload-pages-artifact@v5`, `deploy-pages@v5`; `.github/workflows/pages.yml:24,25,47,59`), not SHAs. There is no Dependabot config. Rust: `cargo audit` shows 1 unmaintained transitive crate (paste).

- **Blocks mainnet: NO**

6.6 Transaction simulation and preview shown to users

- **Covers:** What the user sees before signing: contract, method, amounts, min-out, fees, auth entries, and simulated balance changes.
- **Pass looks like:** For in-app keys, a confirm dialog decoded from the **prepared** transaction (not from UI state) that shows the contract name, function, token amounts, min-out, network, and fee. Freightier users see Freightier's own review screen as well.
- **Evidence that should exist:** UI screenshots and tests that the preview matches the XDR.
- **Current status: Partial.** Transactions *are* simulated (`soroban.prepareTransaction`, `frontend/src/lib/soroban.ts:47`). However, in-app keys sign immediately with no preview (**F-11**, `signer.ts:40-45`). Freightier provides its own prompt.
- **Blocks mainnet: YES**

6.7 Network passphrase checks (a mainnet build can't point at testnet, or the reverse)

- **Covers:** Build-time network selection, runtime `getNetwork()` verification against the RPC, Freightier network mismatch handling, and contract IDs per network.
- **Pass looks like:** Separate build targets. At startup the app calls RPC `getNetwork` and **refuses to operate** if the passphrase differs from the build. A Freightier mismatch **blocks signing**, not just shows a pill. Contract IDs are loaded from a per-network file with a checksum.
- **Evidence that should exist:** Tests for each mismatch case.
- **Current status: Partial.** The frontend hard-codes `Networks.TESTNET` and throws if the deployment file or `VITE_NETWORK` disagrees (`frontend/src/lib/config.ts:13-20`). Transactions are built with that passphrase, so signatures can't be replayed on another network. But `VITE_RPC_URL/VITE_HORIZON_URL` can override the endpoints without a runtime `getNetwork` check. A Freightier mismatch only sets a warning pill (`wallet.tsx:84-85`, `components/Layout.tsx:47`). There is no mainnet build path at all (see §8.3). The bot has `assertTestnet` (`bot/src/soroban.ts:33-36`), a good pattern to copy.
- **Blocks mainnet: YES**

6.8 Phishing protections

- **Covers:** An official domain, a published list of contract IDs, a signed release, anti-phishing wording, `?ref=` handling, and lookalike-domain monitoring.
- **Pass looks like:** Contract IDs and the canonical URL are published in several places (repo, docs, socials). The app shows the verified domain. Referral links can't alter anything beyond attribution. Lookalike domains are monitored.
- **Evidence that should exist:** A security page and a domain-monitoring setup.
- **Current status: Not started.** `?ref=` is auto-captured into `localStorage` (regex-validated, first-write-wins, `frontend/src/App.tsx:15-21`). That is low risk but should be disclosed.
- **Blocks mainnet: YES**

6.9 Custom domain and HTTPS (github.io vs your own domain)

- **Covers:** A dedicated origin for key isolation, header control (CSP), DNSSEC/CAA, HSTS preload, and registrar lock.

- **Pass looks like:** An owned domain (e.g. `app.quasaria.xyz`) on a host that can set security headers (Cloudflare Pages/Workers, Netlify, S3+CloudFront), with HTTPS, HSTS preload, a CAA record, and registrar lock with 2FA.
- **Evidence that should exist:** DNS records, a header scan, and registrar settings.
- **Current status: Fail.** The site is served at `https://yogibear1323.github.io/quasaria/`. HTTPS and HSTS are fine, but the origin is shared across all of the owner's Pages sites and custom headers are impossible (**F-16**).
- **Blocks mainnet: YES**

6.10 Mock tokens and testnet artifacts removed

- **Covers:** Mock stablecoins, QUSD, demo pools, friendbot, `DEMO_POOLS` fallbacks, and offline demo mode in a mainnet build.
- **Pass looks like:** The mainnet build contains no mock asset, no friendbot call, and no demo-data fallback for prices or quotes. There is a CI check for this.
- **Evidence that should exist:** A build-time grep test and the mainnet config file.
- **Current status: Fail.** 20 of the 21 stablecoin pools are mock tokens (`deployments/testnet-stablecoins.json`, `frontend/src/config/testnet-stablecoins.json`). QUSD is admin-issued (`scripts/deploy-testnet.sh:65`). There is a friendbot call (`keys.ts:128`), a `DEMO_POOLS` fallback (`Trade.tsx:69-78`) and an `OFFLINE_DEMO` mode (`frontend/src/lib/config.ts:38`).
- **Blocks mainnet: YES**

6.11 Accuracy of product claims

- **Covers:** Leverage, APR/APY, “backed”, “verified issuer” and other marketing claims matching the code and config.
- **Pass looks like:** Every number in the UI is read from chain or config, and marketing copy has been reviewed (see 9.9).
- **Evidence that should exist:** A claims-to-source table.
- **Current status: Fail. F-15:** the landing page shows 20× (`Landing.tsx:471`; `docs/landing-outline.md:45`) while the deployment caps at 10× (`deploy-testnet.sh:90`; `README :231, :622` mention the 20× contract hard cap). The “✓ Allowlisted issuer” badge (`Landing.tsx:258`) may read as verification.
- **Blocks mainnet: YES**

6.12 Market-data snapshot (stellarchain.io)

- **Covers:** The static snapshot `frontend/public/data/stellarchain-snapshot.json`, its freshness, labeling, fallbacks, and the fact that it never feeds on-chain logic.
- **Pass looks like:** Clearly labeled “as of” time, a staleness banner, never used for transaction amounts, and API terms of use checked.
- **Evidence that should exist:** UI labels and a terms review.
- **Current status: Partial.** The snapshot was generated 2026-09-26 14:25 UTC (07:25 MST). The loader has a staleness flag and a Horizon fallback (`frontend/src/lib/markets.ts:42-43`, `stellarchain.ts`). It isn't used for transaction amounts. The stellarchain.io API terms haven't been reviewed.
- **Blocks mainnet: NO**

7. Bot / keeper

7.1 Keeper and bot key handling

- **Covers:** Where `QUASARIA_SECRET` lives, its privileges, funding, rotation, and the collateral trustline.
- **Pass looks like:** A dedicated keeper account with minimal XLM, no admin power, and the secret held in a secrets manager or KMS (never in plaintext `.env` on disk). A trustline to the collateral asset (USDC on mainnet) so liquidation bonuses can be paid. User bot keys are delegated operators that can't withdraw.
- **Evidence that should exist:** A deployment manifest, secrets-manager configuration, and a rotation procedure.
- **Current status: Partial.** The operator design is least-privilege (trade, no withdraw; `leverage-vault/src/lib.rs:17,419`). The secret is read from the environment (`bot/src/index.ts:65,72`), with `bot/.env` git-ignored and no secret committed (secret scan clean). It is still plaintext in `.env`. No trustline/funding runbook exists.
- **Blocks mainnet: YES**

7.2 Liveness monitoring

- **Covers:** Heartbeats, sweep-duration metrics, the count of liquidatable positions, keeper balance, and pager alerts.
- **Pass looks like:** A health endpoint or heartbeat to an external monitor, a Prometheus/OpenTelemetry metric per sweep, and alerts for a missed sweep, a failing transaction, a low balance, or a stale oracle.
- **Evidence that should exist:** Dashboards, alert rules, and an on-call rota.
- **Current status: Not started.** Only `console.log/console.error` (`bot/src/index.ts:83-87`). In strategy mode, keeper errors are swallowed (`:106`) (**F-20**).
- **Blocks mainnet: YES**

7.3 Liquidation keeper reliability

- **Covers:** Redundancy, fee bidding under congestion, retries, O(N) scaling, handling stale oracles or archived entries, and incentives.
- **Pass looks like:** At least 2 independent keepers (different hosts, RPCs and keys) plus open-source keeper docs for third parties. Dynamic fees (`getFeeStats`) with retries and backoff. Batched reads. Sweep time under the oracle resolution at 10x the expected positions. Graceful handling of `Healthy/StalePrice` errors.
- **Evidence that should exist:** A load-test report and a failover drill.
- **Current status: Partial.** The decision logic is pure and unit-tested (`bot/src/keeper.ts:8-20`; 20/20 bot tests pass), and it has a dry-run mode (`:53`). Gaps: a single loop; a fixed `BASE_FEE` (`bot/src/soroban.ts:50`); no retry; sequential per-id reads (`keeper.ts:29`); no incentive for underwater positions (bonus 0, §2.7); and it depends on a fresh oracle (**F-06**) (**F-20**).
- **Blocks mainnet: YES**

7.4 Network guards and mainnet mode

- **Covers:** Refusing the wrong network, and having an explicit mainnet configuration.
- **Pass looks like:** A config `network` enum with a runtime RPC `getNetwork` passphrase check for each network.

- **Evidence that should exist:** Tests.
- **Current status: Partial.** Config rejects anything except `testnet` (`bot/src/config.ts:39`), and runtime `assertTestnet` checks the RPC passphrase (`bot/src/soroban.ts:33-36`). A good guard, but there is no mainnet mode yet (by design).
- **Blocks mainnet:** NO

7.5 User trading bots (scope decision)

- **Covers:** Whether user strategy bots ship at mainnet launch; their risk limits and disclosures.
- **Pass looks like:** Either deferred, or shipped with hard risk caps, a kill switch, and disclosures reviewed by counsel (automated leveraged trading raises regulatory questions; see 9.2).
- **Evidence that should exist:** A scope decision record.
- **Current status: Partial.** Risk caps exist (`maxLeverage`, `maxOpenPositions`, `maxDailyLossPct`, `bot/src/config.ts:40-47`), but no scope decision is recorded.
- **Blocks mainnet:** NO

8. Deployment and readiness

8.1 Reproducible wasm builds with published hashes

- **Covers:** Pinned toolchain and CLI, a deterministic build, and SHA-256 hashes published per release and matched to the on-chain wasm hash.
- **Pass looks like:** `rust-toolchain.toml` plus a pinned `stellar-cli` version (or a Docker image by digest). A CI job builds and publishes the hashes in the release notes. A script compares them with `stellar contract fetch` output.
- **Evidence that should exist:** `deployments/<network>.hashes.json`, a CI artifact, and a signed release tag.
- **Current status: Partial.** This review rebuilt all 7 contracts with `stellar-cli` 28.0.0 (tarball digest verified) and `rustc` 1.98.1. The local hashes matched every one of the 8 active testnet contracts on-chain (\$0.3). There is no pinned toolchain file, no hashes file in `deployments/`, and no CI build.
- **Blocks mainnet:** YES

8.2 Mainnet deploy script with dry-run (multisig-aware)

- **Covers:** A scripted, reviewable mainnet deployment: build → verify hashes → upload → deploy with multisig admin → wire → verify config → hand-off.
- **Pass looks like:** `scripts/deploy-mainnet.sh` (or a TypeScript equivalent) with `DRY_RUN=1` default. It **builds unsigned XDR for multisig signing** (no hot key). It asserts the network passphrase, the native SAC ID, and the Reflector ID. It refuses the mock oracle and legacy hashes, and writes `deployments/mainnet.json`. Rehearsed end-to-end on testnet under the same multisig setup.
- **Evidence that should exist:** The script, a rehearsal log, and a reviewer sign-off.
- **Current status: Not started.** `scripts/deploy-testnet.sh` refuses non-testnet (`:21`) and supports `DRY_RUN`, which is a good base. But it signs with a single CLI identity, deploys the mock oracle and QUSD, and writes `.env` files.
- **Blocks mainnet:** YES

8.3 Config switch (testnet / mainnet)

- **Covers:** A single source of truth for network, RPC, contract IDs and assets across the frontend, bot and scripts.
- **Pass looks like:** `deployments/{testnet,mainnet}.json` selected at build time. Mainnet build artifacts are separate and CI-checked (no testnet IDs or passphrase in the mainnet bundle, and the reverse).
- **Evidence that should exist:** Build configuration and a CI grep test.
- **Current status: Fail.** Testnet is hard-coded in `frontend/src/lib/config.ts:13-20`, `bot/src/config.ts:39`, `bot/src/soroban.ts` (`Networks.TESTNET`), and every script. There is no mainnet configuration path.
- **Blocks mainnet: YES**

8.4 Contract verification

- **Covers:** Letting users check that the on-chain wasm matches the source: published hashes and source links, and explorer verification where available (e.g. stellar.expert contract validation via GitHub Actions attestations).
- **Pass looks like:** Each mainnet contract ID links to its source commit and hash. Explorer verification is done where supported.
- **Evidence that should exist:** A verification page and explorer links.
- **Current status: Partial.** Hashes are reproducible (§0.3), and `deployments/testnet.json` lists the IDs. Nothing is published or verified on an explorer.
- **Blocks mainnet: NO**

8.5 Monitoring and alerting

- **Covers:** On-chain monitoring of events (admin calls, large withdrawals, liquidations, oracle age, reserve runway, TVL, QFX backing ratio, TTL expiry) plus frontend uptime.
- **Pass looks like:** Event indexer and alert rules sent to a pager/Slack. Alert on **any** admin function call, on backing < 100%, oracle age > 2× resolution, reserve runway < 30 days, keeper failures, or TTL below threshold.
- **Evidence that should exist:** Alert rules in the repo and dashboard links.
- **Current status: Not started.**
- **Blocks mainnet: YES**

8.6 Incident-response runbook

- **Covers:** Roles, severity levels, contact tree, decision authority, pause/drain procedures, comms templates, the post-mortem template, and coordination with SDF, Reflector and Circle.
- **Pass looks like:** `docs/runbooks/incident-response.md`, rehearsed (tabletop exercise) before launch.
- **Evidence that should exist:** The runbook and a drill log.
- **Current status: Not started.**
- **Blocks mainnet: YES**

8.7 Status page

- **Covers:** Public status for the frontend, RPC, keeper, oracle freshness, and incidents.
- **Pass looks like:** A hosted status page on a separate domain/provider, linked from the app.
- **Evidence that should exist:** The URL.

- **Current status: Not started.**
- **Blocks mainnet: NO**

8.8 Rollback and pause plan

- **Covers:** Given immutable contracts: how to stop new risk, protect users and migrate (frontend kill switch, pausing via guardian, redeploying, migration tooling, communication).
- **Pass looks like:** A documented plan for each contract, with tested pause (5.5), a frontend “withdraw-only” mode, and a migration script template.
- **Evidence that should exist:** The plan document and drill log.
- **Current status: Not started.** There is no pause (**F-13**) and no upgrade path (§1.11). `scripts/migrate-qfx-peg-testnet.sh` is the only migration precedent.
- **Blocks mainnet: YES**

8.9 Continuous integration

- **Covers:** CI for contract tests, clippy, `cargo audit`, Scout, coverage, the wasm build and hashes, frontend and bot tests, and `npm audit`.
- **Pass looks like:** Every PR runs everything above, with branch protection and required reviews.
- **Evidence that should exist:** Workflow files and a branch-protection screenshot.
- **Current status: Partial.** The only workflow is `.github/workflows/pages.yml`, which builds and deploys the frontend. It does not run contract or bot tests.
- **Blocks mainnet: NO** (it is needed for 1.3/1.5/8.1 evidence)

8.10 Testnet soak of the final (audited) code

- **Covers:** Running the exact audited build on testnet for a few weeks with real users, keepers and a live oracle.
- **Pass looks like:** 2–4 weeks with no Sev-1/2 incidents, all monitoring green, and the keeper and oracle running continuously.
- **Evidence that should exist:** A soak report.
- **Current status: Partial.** The current code is live on testnet and matches its hashes, but it is not the final code. The oracle is stale (**F-06**), and the vault can’t operate.
- **Blocks mainnet: YES**

9. Legal, compliance and business

Every item in this section needs review by a qualified lawyer (ideally crypto/fintech counsel in each target jurisdiction). Nothing below is legal advice or a legal conclusion. It records only what exists in the repo and which questions need answering.

9.1 Legal entity and jurisdiction

- **Covers:** Which entity operates the frontend, holds the admin keys and treasury, and receives fees. Where it is incorporated. Directors’ liability.

- **Pass looks like:** Counsel-approved entity structure and jurisdiction memo. Clear separation of the entity from the open-source code.
- **Evidence that should exist:** Incorporation documents and a counsel memo.
- **Current status: Not started.** Nothing in the repo identifies an operating entity. *Needs a qualified lawyer.*
- **Blocks mainnet: YES**

9.2 Securities, derivatives and commodities exposure

- **Covers:** Leveraged perpetual-style trading (the vault is a counterparty to leveraged positions), holder yield on QFX, staking rewards, referral rewards, and automated trading bots. These product types are commonly treated as **high regulatory risk** in many jurisdictions.
- **Pass looks like:** A counsel memo per product and per target jurisdiction. Products or regions are restricted accordingly (e.g., geofencing, product removal).
- **Evidence that should exist:** Counsel memos and the resulting product and geo decisions.
- **Current status: Not started.** *Needs a qualified lawyer.* Technically, launching without the vault and the bots (the recommendation in §12) also shrinks this exposure; whether that is enough is for counsel to decide.
- **Blocks mainnet: YES**

9.3 KYC/AML and sanctions stance

- **Covers:** Whether the operator must do KYC, sanctions screening of addresses, IP geoblocking of sanctioned regions, and the obligations of fiat-backed stablecoin issuers (Circle can freeze USDC/EURC).
- **Pass looks like:** A documented, counsel-approved stance and any controls it requires.
- **Evidence that should exist:** A compliance policy.
- **Current status: Not started.** *Needs a qualified lawyer.*
- **Blocks mainnet: YES**

9.4 Terms of service

- **Covers:** Terms for using the interface: non-custodial nature, no warranty, eligibility, prohibited jurisdictions, limitation of liability, and dispute resolution.
- **Pass looks like:** Counsel-drafted ToS, accepted in the app before first use.
- **Evidence that should exist:** `/terms` page and a record of acceptance in the UI.
- **Current status: Not started.** There is no ToS in the repo or on the site. *Needs a qualified lawyer.*
- **Blocks mainnet: YES**

9.5 Privacy policy

- **Covers:** Data processing: localStorage (encrypted key, `?ref=` referrer, market cache), and IP addresses sent to third parties (the Stellar RPC/Horizon provider, stellarchain.io, friendbot on testnet, GitHub Pages). GDPR/CCPA applicability.
- **Pass looks like:** A counsel-reviewed privacy policy listing every processor, and a cookie/storage notice where required.
- **Evidence that should exist:** `/privacy` page.
- **Current status: Not started.** Storage keys: `quasaria.keystore.v1` (`keys.ts:69`), `REF_KEY` (`App.tsx:19`), and the stellarchain cache (`stellarchain.ts:238`). *Needs a qualified lawyer.*

- **Blocks mainnet: YES**

9.6 Risk disclosures

- **Covers:** Leverage/liquidation risk, smart-contract risk, oracle risk, admin-key risk, reserve depletion (yield can stop), impermanent loss, stablecoin depeg or freeze, and “unaudited” status until audited.
- **Pass looks like:** Counsel-reviewed disclosures shown before each high-risk action (first leverage trade, first QFX deposit, LP deposit), with acknowledgement.
- **Evidence that should exist:** Disclosure texts and UI flow screenshots.
- **Current status: Partial.** There are warnings in `Landing.tsx:463`, `:568` and README lines 3, 23 and 741 (“testnet only / unaudited”). There is no per-action acknowledgement, no disclosure of admin powers (§5.1), and no disclosure of silent profit caps (F-01) or yield stopping (F-13). *Needs a qualified lawyer.*
- **Blocks mainnet: YES**

9.7 Stablecoin issuer terms and trademarks

- **Covers:** Circle’s terms for USDC/EURC use, trademark and logo use, verifying the real mainnet issuers, and the name/branding of “QUSD” or any in-house stable asset.
- **Pass looks like:** Issuer terms reviewed, trademarks used within guidelines, and no in-house “USD” branded token unless counsel approves.
- **Evidence that should exist:** A counsel note and the verified issuer allowlist (4.6).
- **Current status: Not started.** *Needs a qualified lawyer.*
- **Blocks mainnet: YES**

9.8 Tax

- **Covers:** Entity tax on fees and treasury, user-facing tax information, and any reporting obligations (e.g., broker reporting regimes).
- **Pass looks like:** Tax advisor memo.
- **Evidence that should exist:** The memo.
- **Current status: Unknown.** Nothing in the repo. *Needs a qualified tax advisor/lawyer.*
- **Blocks mainnet: NO** (it should be done before revenue accrues)

9.9 Marketing and claims review

- **Covers:** APR/APY statements, “fully backed”, leverage claims, “allowlisted issuer” badges, and referral promotions.
- **Pass looks like:** Counsel sign-off on all public copy. Every numeric claim is sourced from chain or config (6.11).
- **Evidence that should exist:** Review log.
- **Current status: Not started.** See the known 20× vs 10× mismatch (**F-15**). *Needs a qualified lawyer.*
- **Blocks mainnet: YES**

9.10 Open-source licensing

- **Covers:** Project license, dependency licenses, and attribution.
- **Pass looks like:** License declared everywhere and a dependency license scan (`cargo deny`, `license-checker`) with no incompatible licenses.

- **Evidence that should exist:** Scan report.
- **Current status: Partial.** `LICENSE` is Apache-2.0, and contracts use `license.workspace = true`. No dependency license scan has been run.
- **Blocks mainnet:** NO

10. Funding plan (90-day launch runway)

Assumptions: - XLM = **\$0.217776**, from the repo snapshot `frontend/public/data/stellarchain-snapshot.json`, generatedAt 2026-09-26T14:25:43Z (07:25 MST). - Current on-chain parameters from §0.4: QFX APR 12% (1,200 bps, `deploy-testnet.sh`); staking emissions 8.64 and 17.28 QFX/day; AMM fee 30 bps; vault 10×/5% maintenance. - A **+25% buffer** is applied to consumable reserves. - These are planning numbers, not a treasury approval.

10.1 Holder-yield reserve and eligible-supply cap

- **Covers:** The XLM reserve needed to pay the advertised QFX APR for 90 days, and the cap that keeps the liability bounded.
- **Pass looks like:** An on-chain (or at least frontend-enforced, then on-chain) cap on eligible supply E , with a reserve $\geq E \times 90\text{-day growth} \times 1.25$, funded in tranches (the reserve can't be withdrawn: F-13).
- **Evidence that should exist:** A treasury approval, the funding tx hash, and a cap parameter.
- **Current status: Not started.** There is no cap in code, and the testnet reserve of 2,500 XLM covers only $E \leq 83,261$ QFX for 90 days.
- **Blocks mainnet: YES**

Calculation: - Yield compounds daily (`daily_factor`, `reward-token/src/lib.rs:262`). - 90-day growth = $(1 + 0.12/365)^{90} - 1 = 3.0026\%$ of eligible supply (simple interest would be 2.959%). - At the 25% maximum APR (`MAX_APR_BPS`), growth is 6.356%.

Eligible supply E (QFX = XLM)	90-day payout (XLM)	With +25% buffer (XLM)	≈ USD at \$0.2178	≈ USD at \$0.15 / \$0.30
100,000	3,003	3,753	\$817	\$563 / \$1,126
1,000,000	30,026	37,533	\$8,174	\$5,630 / \$11,260
5,000,000	150,131	187,663	\$40,868	\$28,149 / \$56,299
10,000,000	300,261	375,327	\$81,736	\$56,299 / \$112,598

F-03 (day-boundary sniping) raises the *effective* E above the time-weighted average. Fix F-03 before using this table.

10.2 Staking reward reserves

- **Covers:** The QFX reward reserve per staking pool for 90 days at the current fixed emission.
- **Pass looks like:** Each pool is funded for ≥ 90 days plus a buffer, with a runway alert at 30 days.
- **Evidence that should exist:** Funding tx hashes.
- **Current status: Not started** (mainnet). Testnet pool 1 has 86.6 days of runway (\$2.3).
- **Blocks mainnet:** NO

Calculation: - Pool 0: $8.64 \text{ QFX/day} \times 90 = 777.6$. - Pool 1: $17.28 \times 90 = 1,555.2$. - Total **2,332.8 QFX** (= XLM, since QFX is 1:1), or **2,916 XLM** with the buffer ($\approx \$635$). - At the current rate, emissions are fixed per second, so the APR falls as TVL grows. For example, paying **10% APR on 1,000,000 staked** needs $1,000,000 \times 0.10 / 31,536,000 \times 10^7 \approx 31,710$ stroops/s, which is **24,658 XLM per 90 days**. Size the rate to a target APR at the expected TVL, not the other way round.

10.3 Protocol-owned liquidity (AMM pools)

- **Covers:** The capital needed to seed the launch pools deep enough for usable trade sizes.
- **Pass looks like:** Depth chosen from a target trade size. LP shares held by the treasury multisig.
- **Evidence that should exist:** Liquidity plan (4.1) and treasury approval.
- **Current status: Not started.**
- **Blocks mainnet: YES**

Calculation ($x \cdot y = k$): For a trade to execute at most 1% worse than spot (before the 0.30% fee), trade size $\approx D \times (1/0.99 - 1) = D \times 0.0101$, where D is per-side depth. The spot price then moves about 2%.

Per-side depth	Trade with 1% price impact	XLM needed for the XLM side at \$0.2178	at \$0.15 / \$0.30
\$25,000	$\approx \$253$	114,797 XLM	166,667 / 83,333
\$50,000	$\approx \$505$	229,594 XLM	333,333 / 166,667
\$100,000	$\approx \$1,010$	459,187 XLM	666,667 / 333,333
\$250,000	$\approx \$2,525$	1,147,969 XLM	1,666,667 / 833,333

POL is recoverable capital (subject to impermanent loss), not an expense.

10.4 Leverage-vault counterparty reserve

- **Covers:** The collateral (USDC on mainnet) the vault needs to pay trader profits without truncation.
- **Pass looks like:** Reserve $\geq$ (net open-interest cap) $\times$ (stress move), with OI caps enforced on-chain (§2.8).
- **Evidence that should exist:** Risk parameter document.
- **Current status: Not started.** Testnet has 200,000.52 QUSD (a mock token).
- **Blocks mainnet: YES** (if the vault launches; the recommendation is to **defer the vault**)

Calculation: Assume a 40% adverse move on net OI. A \$50,000 net OI cap needs **\$20,000**. A \$250,000 cap needs **\$100,000**. At 10 $\times$ leverage, \$50k of OI is only \$5k of trader margin, which shows why the reserve and caps must be sized together.

10.5 Operations and security budget

- **Covers:** Keeper XLM, account reserves and storage rent/TTL, oracle upkeep, audit, bounty, monitoring, legal.
- **Pass looks like:** An approved budget line for each item.
- **Evidence that should exist:** Budget sheet.

- **Current status: Not started.**
- **Blocks mainnet: YES**

Estimates: - Keeper: about 1,000 XLM (fees and reserves; assumption). - Rent, TTL extensions and account reserves: about 500 XLM (assumption; re-estimate from `simulateTransaction` resource fees). - Reflector upkeep/subscription: **get a quote**. - Audit: see §12 (typical ranges, not quotes). - Bug bounty pool: typically sized to TVL (see 1.12). - Legal: **get quotes**.

Illustrative 90-day base case: vault **off**, bots **off**, eligible-supply cap $E = 1,000,000$ QFX, current staking rates, two pools (XLM/USDC and QFX/USDC) at \$50k per side.

Line	XLM	USDC	Nature
Holder-yield reserve ($E = 1M$, +25%)	37,533	—	consumed
Staking reserves (+25%)	2,916	—	consumed
Keeper + rent/TTL (assumed)	1,500	—	mostly consumed
XLM/USDC pool, \$50k per side	229,594	50,000	recoverable (IL risk)
QFX/USDC pool, \$50k per side (QFX minted 1:1 from XLM)	229,594	50,000	recoverable (IL risk)
Total	≈ 501,137 XLM	100,000 USDC	≈ \$209k at \$0.2178

Of this, only about **41,949 XLM (≈ \$9.1k)** is consumed over 90 days. The rest is recoverable liquidity. Audit, legal and bounty costs are extra.

11. Additional launch items

11.1 Documentation and specification

- **Covers:** User docs, a protocol specification (formulas, parameters, admin powers), and developer docs for integrators and keepers.
- **Pass looks like:** A spec that auditors can check the code against, and user docs covering every risk and admin power.
- **Evidence that should exist:** `docs/spec/*.md`.
- **Current status: Partial.** README is extensive, and `docs/` has design notes (e.g., `docs/landing-outline.md`). There is no formal spec, and the 20× claim is inconsistent (F-15).
- **Blocks mainnet: NO**

11.2 Communications and governance transparency

- **Covers:** Publishing admin addresses, signer identities or count, timelock queue, parameter change log, and launch announcements.
- **Pass looks like:** A public “governance and admin” page and a changelog of on-chain parameter changes.
- **Evidence that should exist:** The page.
- **Current status: Not started.**
- **Blocks mainnet: NO**

11.3 Wallet compatibility

- **Covers:** Freightner, hardware wallets through Freightner, and optionally other wallets (e.g., via Stellar Wallets Kit), including auth-entry signing for multi-party flows.
- **Pass looks like:** Tested with at least Freightner + Ledger on mainnet. Clear guidance to prefer an external wallet for material balances.
- **Evidence that should exist:** A test matrix.
- **Current status: Partial.** Freightner works (`frontend/src/lib/wallet.tsx`, `FreightnerSigner`), and in-app keys work. There is no hardware-wallet test and no other wallets.
- **Blocks mainnet:** NO

12. Executive summary

12.1 Verdict: NO-GO for mainnet

Quasaria is a well-structured testnet scaffold. Its QFX peg design is sound: 1:1 backed, no admin mint, confirmed on-chain. Router slippage and deadline checks exist, user functions authenticate properly, dependency audits are clean, and builds are reproducible and match the chain. It is **not** ready to hold real funds:

- **No external audit.** Contracts are self-labeled “testnet-only scaffold, unaudited”.
- **Confirmed contract bugs with PoCs:** a vault DoS through unbounded storage plus dust positions that can’t be liquidated (F-02), QFX yield sniping (F-03), a staking overflow that bricks pools (F-04), and future-dated oracle prices being accepted (F-07). Storage-TTL gaps (F-08) come on top.
- **Full admin centralization:** one hot key with thresholds 0/0/0 controls all 7 contracts (F-09). There is no multisig, timelock, pause or upgrade path (F-13). The admin can take the entire vault reserve or re-point its oracle instantly (F-01), and can drain AMM pools through a malicious referral registry (F-05).
- **The oracle is an admin-pushed mock**, currently 17.4 h stale, which halts the vault (F-06). There are no deviation bounds and no fallback.
- **Economic design gaps:** the vault has no OI caps, funding or spread and silently truncates profits (F-11/F-14). The holder-yield liability is unbounded (F-13).
- **No legal work** (entity, product classification, ToS, privacy, disclosures).

12.2 Status counts

Area	Items	Pass	Partial	Fail	Not started	Unknown	Block mainnet
1. Smart-contract security	12	1	4	3	4	0	9
2. Economics / tokenomics	10	0	5	3	2	0	5
3. Oracle	5	0	1	1	3	0	4
4. Liquidity, staking, pools	6	0	5	0	1	0	4
5. Access control & admin keys	8	1	3	1	3	0	5
6. Frontend & wallet	12	2	5	4	1	0	8
7. Bot / keeper	5	0	4	0	1	0	3

Area	Items	Pass	Partial	Fail	Not started	Unknown	Block mainnet
8. Deployment & readiness	10	0	4	1	5	0	7
9. Legal / compliance	10	0	2	0	7	1	8
10. Funding plan	5	0	0	0	5	0	4
11. Additional	3	0	2	0	1	0	0
Total	86	4	35	13	33	1	57

Items that block mainnet (57): 1.1 (Not started), 1.2 (Partial), 1.4 (Not started), 1.6 (Fail), 1.7 (Partial), 1.9 (Fail), 1.10 (Fail), 1.11 (Not started), 1.12 (Not started), 2.2 (Fail), 2.6 (Fail), 2.7 (Partial), 2.8 (Fail), 2.10 (Not started), 3.1 (Fail), 3.2 (Partial), 3.3 (Not started), 3.4 (Not started), 4.1 (Not started), 4.3 (Partial), 4.4 (Partial), 4.6 (Partial), 5.2 (Fail), 5.3 (Not started), 5.4 (Not started), 5.5 (Not started), 5.6 (Partial), 6.2 (Partial), 6.3 (Fail), 6.6 (Partial), 6.7 (Partial), 6.8 (Not started), 6.9 (Fail), 6.10 (Fail), 6.11 (Fail), 7.1 (Partial), 7.2 (Not started), 7.3 (Partial), 8.1 (Partial), 8.2 (Not started), 8.3 (Fail), 8.5 (Not started), 8.6 (Not started), 8.8 (Not started), 8.10 (Partial), 9.1 (Not started), 9.2 (Not started), 9.3 (Not started), 9.4 (Not started), 9.5 (Not started), 9.6 (Partial), 9.7 (Not started), 9.9 (Not started), 10.1 (Not started), 10.3 (Not started), 10.4 (Not started), 10.5 (Not started).

Status of the blocking items: Pass 0, Partial 16, Fail 13, Not started 28, Unknown 0.

Full item index

#	Item	Status	Blocks mainnet
1.1	External audit by a recognized Soroban auditor	Not started	YES
1.2	Internal security review and threat model	Partial	YES
1.3	Static analysis (clippy, cargo-audit, Scout)	Partial	no
1.4	Fuzz and property/invariant tests	Not started	YES
1.5	Test coverage	Partial	no
1.6	Arithmetic overflow and rounding	Fail	YES
1.7	Reentrancy and cross-contract call safety	Partial	YES
1.8	Authorization (<code>require_auth</code>) on every state-changing function	Pass	no
1.9	Storage TTL and archival handling	Fail	YES
1.10	Unbounded storage, loops and DoS	Fail	YES
1.11	Upgradeability and migration policy	Not started	YES
1.12	Bug bounty and security contact	Not started	YES
2.1	QFX peg invariant (1 QFX = 1 XLM, fully backed)	Partial	no
2.2	Holder-yield reserve sustainability	Fail	YES
2.3	Staking reward runway	Partial	no
2.4	Referral fee split	Partial	no
2.5	AMM fee and constant-product invariant	Partial	no

#	Item	Status	Blocks mainnet
2.6	Manipulation vectors (flash-style swaps, oracle and pool price manipulation)	Fail	YES
2.7	Leverage-vault liquidation math	Partial	YES
2.8	Bad debt, solvency and open-interest limits	Fail	YES
2.9	Insurance / backstop	Not started	no
2.10	Economic stress testing	Not started	YES
3.1	Replace the mock oracle with a production oracle	Fail	YES
3.2	Staleness checks	Partial	YES
3.3	Deviation bounds / sanity checks	Not started	YES
3.4	Fallback oracle	Not started	YES
3.5	Oracle operations	Not started	no
4.1	Initial liquidity plan	Not started	YES
4.2	Minimum liquidity and first-depositor attack	Partial	no
4.3	Slippage and deadline protection (contract and UI)	Partial	YES
4.4	Withdrawal paths under stress	Partial	YES
4.5	Staking lifecycle	Partial	no
4.6	Asset listing and issuer verification (USDC/EURC etc.)	Partial	YES
5.1	Privileged-function inventory (from the code)	Pass	no
5.2	Key custody (who holds the keys, hardware wallets)	Fail	YES
5.3	Multisig plan (Stellar account thresholds)	Not started	YES
5.4	Timelocks	Not started	YES
5.5	Emergency pause / guardian	Not started	YES
5.6	Rug vectors and free-mint paths	Partial	YES
5.7	Admin rotation	Partial	no
5.8	Role separation	Partial	no
6.1	Key-generation entropy	Pass	no
6.2	Remember-me key storage and encryption	Partial	YES
6.3	CSP, XSS and clickjacking	Fail	YES
6.4	Dependency audit (npm audit)	Pass	no
6.5	Supply-chain risk	Partial	no
6.6	Transaction simulation and preview shown to users	Partial	YES
6.7	Network passphrase checks (a mainnet build can't point at testnet, or the reverse)	Partial	YES

#	Item	Status	Blocks mainnet
6.8	Phishing protections	Not started	YES
6.9	Custom domain and HTTPS (github.io vs your own domain)	Fail	YES
6.10	Mock tokens and testnet artifacts removed	Fail	YES
6.11	Accuracy of product claims	Fail	YES
6.12	Market-data snapshot (stellarchain.io)	Partial	no
7.1	Keeper and bot key handling	Partial	YES
7.2	Liveness monitoring	Not started	YES
7.3	Liquidation keeper reliability	Partial	YES
7.4	Network guards and mainnet mode	Partial	no
7.5	User trading bots (scope decision)	Partial	no
8.1	Reproducible wasm builds with published hashes	Partial	YES
8.2	Mainnet deploy script with dry-run (multisig-aware)	Not started	YES
8.3	Config switch (testnet / mainnet)	Fail	YES
8.4	Contract verification	Partial	no
8.5	Monitoring and alerting	Not started	YES
8.6	Incident-response runbook	Not started	YES
8.7	Status page	Not started	no
8.8	Rollback and pause plan	Not started	YES
8.9	Continuous integration	Partial	no
8.10	Testnet soak of the final (audited) code	Partial	YES
9.1	Legal entity and jurisdiction	Not started	YES
9.2	Securities, derivatives and commodities exposure	Not started	YES
9.3	KYC/AML and sanctions stance	Not started	YES
9.4	Terms of service	Not started	YES
9.5	Privacy policy	Not started	YES
9.6	Risk disclosures	Partial	YES
9.7	Stablecoin issuer terms and trademarks	Not started	YES
9.8	Tax	Unknown	no
9.9	Marketing and claims review	Not started	YES

#	Item	Status	Blocks mainnet
9.10	Open-source licensing	Partial	no
10.1	Holder-yield reserve and eligible-supply cap	Not started	YES
10.2	Staking reward reserves	Not started	no
10.3	Protocol-owned liquidity (AMM pools)	Not started	YES
10.4	Leverage-vault counterparty reserve	Not started	YES
10.5	Operations and security budget	Not started	YES
11.1	Documentation and specification	Partial	no
11.2	Communications and governance transparency	Not started	no
11.3	Wallet compatibility	Partial	no

12.3 Blockers in priority order

1. **Contract correctness and external audit.** Fix F-02, F-03, F-04, F-07 and F-08, add fuzz/invariant tests (1.4) and raise coverage (1.5), then commission an external Soroban audit plus re-audit (1.1). Items 1.1, 1.2, 1.4, 1.6, 1.7, 1.9, 1.10.
2. **Admin centralization.** Make every admin a multisig account from deploy time (5.2, 5.3). Add a timelock on `set_oracle`, `set_referral`, `withdraw_liquidity`, `config/APR/fee` changes (5.4), add a guardian pause that leaves exits open (5.5), re-cap `share_bps` in the pool (F-05), and decide and document an upgrade policy (1.11). Items 5.2–5.6, 1.11, 8.8.
3. **Oracle.** Integrate Reflector with staleness (including future timestamps), deviation bounds, a fallback and reduce-only exits (3.1–3.4). Never deploy the mock.
4. **Vault economics, or defer the vault.** Add OI, size and skew caps, fees/spread or delay, a keeper incentive for underwater positions, and a profit-cap disclosure (2.6–2.8, 2.10).
Recommendation: launch spot (swap, LP, QFX, staking) first, and ship the vault and user bots only after a separate audit and risk review.
5. **QFX holder-yield liability.** Add an eligible-supply cap and time-weighted accrual, funded in tranches (2.2, 10.1).
6. **Legal and compliance.** Entity, securities/derivatives analysis, KYC/AML stance, ToS, privacy, disclosures, stablecoin terms, marketing review (9.1–9.7, 9.9). *Needs qualified counsel.*
7. **Deployment infrastructure.** Own domain with CSP headers (6.3, 6.9), a mainnet config switch (8.3), a multisig-aware mainnet deploy script with dry-run (8.2), published hashes (8.1), monitoring/alerting (8.5), an incident runbook (8.6), and a rollback/pause plan (8.8).
8. **Frontend safety.** Transaction preview for in-app keys (6.6), nonzero slippage mins and no demo-quote fallback (4.3), a runtime network check that blocks signing (6.7), removal of mock tokens/testnet artifacts (6.10), and corrected 20× and issuer claims (6.11, 4.6), plus phishing measures (6.8) and remember-me origin isolation (6.2).
9. **Keeper.** Redundancy, dynamic fees and retries, monitoring, secret management and a trustline (7.1–7.3).

10. **Funding approvals.** Treasury sign-off for POL, reserves and ops (10.1, 10.3–10.5).

11. **Bug bounty, and a testnet soak of the audited build** (1.12, 8.10).

12.4 Suggested order of work

Phase	Work	Rough duration (estimate)
0: Decide	Launch scope (spot-first; vault and bots deferred). Engage counsel (\$9). Name signers and buy hardware wallets.	1–2 weeks, in parallel with Phase 1
1: Fix and harden	Fix F-01 to F-08, F-10, F-17 to F-19. Add pause/guardian, timelock, TTL bumps, min sizes, caps, and yield time-weighting. Add fuzz/property tests, raise coverage to $\geq 95\%$, add CI (tests, clippy strict, cargo audit, Scout, npm audit, wasm hashes).	3–6 weeks
2: Audit	Write the STRIDE threat model and spec. Apply to the SDF Soroban Audit Bank (if SCF-eligible) or get quotes. Code freeze, audit, remediation, re-audit.	queue + 3–6 weeks (see 12.5)
3: Infrastructure	Custom domain with headers/CSP, mainnet config switch, multisig deploy script with dry-run, Reflector integration, monitoring/alerts, runbooks, status page, keeper HA.	2–4 weeks, in parallel with Phase 2
4: Soak and bounty	Deploy the audited build to testnet under the production multisig setup with live Reflector. 2–4 week soak, public bounty live, tabletop incident drill.	2–4 weeks
5: Guarded launch	Mainnet with TVL, deposit and eligible-supply caps. Raise caps gradually as monitoring stays green. Vault and bots only after their own audit.	ongoing

12.5 External audit: typical industry ranges (NOT quotes)

These are **publicly published typical ranges**, not quotes for Quasaria. **Get written quotes** from Soroban-experienced firms (see `docs/audit-firm-shortlist.md`, which is untracked in this workspace).

Source (public)	What it says
Sherlock, “Smart Contract Audit Pricing: A Market Reference for 2026” (sherlock.xyz/post/smart-contract-audit-pricing-a-market-reference-for-2026)	Most DeFi audits fall between \$25k and \$100k . Mid-complexity protocols typically \$40k–\$100k . Re-audit or fix-review \$5k–\$20k per pass.
Sherlock docs, “Audit pricing and timeline” (docs.sherlock.xyz/audits/protocols/audit-pricing-and-timeline)	Duration scales with nSLOC: about 2,000 nSLOC $\approx$ 12 days , 3,000 $\approx$ 18 days of review (Solidity-oriented).
AuditOne, “Preparation and pricing” (docs.auditone.io/stakeholders/for-clients/preparation-and-pricing.md)	Rust is about \$14k per 1k nSLOC ($\approx$ 2 weeks) ; 3k nSLOC is about \$42k ($\approx$ 4 weeks) .
Procur3, “Smart contract audit cost 2026” (procur3.io/blog/smart-contract-audit-cost-2026)	DeFi protocol audits about \$60k–\$120k including one remediation pass. Rust/non-EVM carries a 30–120% premium . Booking queues 4–12 weeks .
Lollychain, “Smart Contract Audit Cost: 2026 Pricing and Risk Reality” (lollychain.com/articles/smart-contract-audit-cost-worth/)	Mid-size DeFi (about 1,000–3,000 lines of custom logic) \$20k–\$60k, 3–6 weeks .

Source (public)	What it says
SDF Soroban Audit Bank (stellar.org/grants-and-funding/soroban-audit-bank; SCF handbook official rules)	Audit subsidy for SCF-funded projects: a 5% co-pay (refundable) , requires tool-scan results and a STRIDE threat model , with pre-approved firms (e.g., Certora, OtterSec, Runtime Verification, Veridise, among others).

Applied to Quasaria ($\approx$ 2,430 production nSLOC, 7 contracts, Rust/Soroban, with a leverage vault): the public ranges above imply roughly **\$30k-\$120k** and **3-6 weeks** of review, plus a **4-12 week** booking queue and **\$5k-\$20k** per re-audit pass. Deferring the vault (567 nSLOC) and dropping the mock oracle shrinks the scope. **These are typical industry ranges, not quotes. Get quotes.**

12.6 Limits of this review

- This is an internal review by one reviewer, not an audit. Severity ratings are provisional.
- No formal verification or long fuzzing campaigns were run.
- Mainnet was not touched. All live data is from Stellar testnet (read-only).
- No legal conclusions are drawn; §9 lists questions for counsel.
- Funding figures are planning estimates using the XLM price from the repo snapshot and the stated assumptions.